

Machine Learning



Dr. Ahmed Hesham Mostafa
Faculty of Computers and AI
Helwan University
Lecture 2

Course Staff: Instructors (Short Bio)

- **Dr. Ahmed Hesham Mostafa**
- Bachelor in Computer Science *from* Helwan University
- Masters in Computer Science (Malware Analysis & Artificial Intelligence) *from* Helwan University
- PhD. in Computer Science (Computer Vision & Artificial Intelligence) *from* Helwan University
- E-mail : ahmedheshamostafa@gmail.com

Linear Regression in Machine Learning

- Linear Regression is a fundamental statistical method used to define linear relationship between a dependent and one or more independent variables.
- It helps predict outcomes by fitting a straight line to observed data points, making it easy to interpret and apply.

What is Linear Regression in Machine Learning?

- Linear Regression is a supervised learning algorithm that is used to model the relationship between a dependent variable and an independent variable.
- The algorithm finds the best fit straight line relationship (linear equation) between the two variables.
- This statistical method is then used to predict the outcome of future events and is quite useful for predictive analysis.

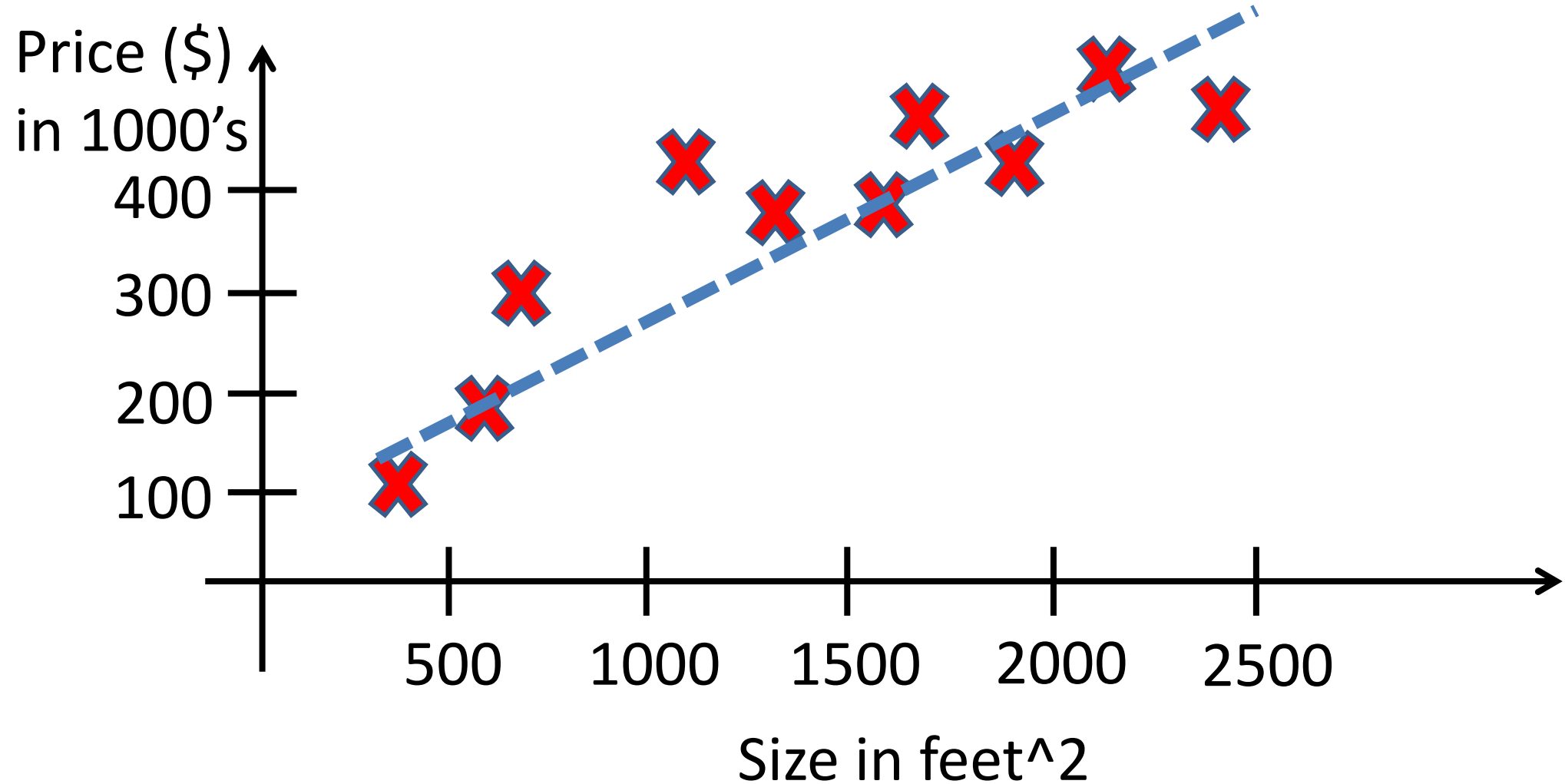
What is Linear Regression in Machine Learning?

- **For example** we want to predict a student's exam score based on how many hours they studied. We observe that as students study more hours, their scores go up. In the example of predicting exam scores based on hours studied. Here
 - **Independent variable (input):** Hours studied because it's the factor we control or observe.
 - **Dependent variable (output):** Exam score because it depends on how many hours were studied.
- We use the independent variable to predict the dependent variable.

What makes Linear Regression so Important?

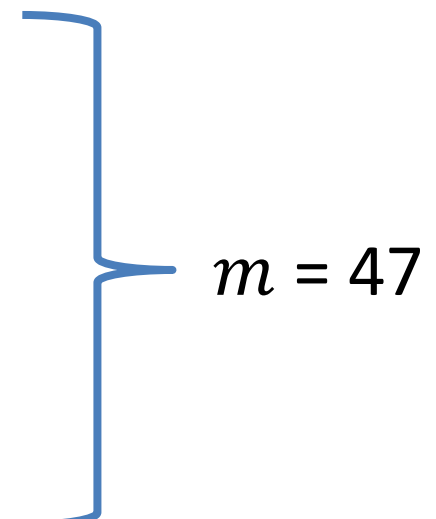
- This algorithm acts as a starting point for machine learning, making it quite simple to understand data.
- It helps in predictive analysis proving useful in many industries including marketing, finance, and even healthcare.
- It is the base for many advanced algorithms like logistic regression or neural networks.
- It is computationally efficient making it the most widely used statistical technique for regression tasks.
- The technique offers great performance in tasks for which the relationship between the variables is roughly linear

House pricing prediction



Training set

Size in feet ² (x)	Price (\$) in 1000's (y)
2104	460
1416	232
1534	315
852	178
...	...



- Notation:

- m = Number of training examples
- x = Input variable / features
- y = Output variable / target variable
- (x, y) = One training example
- $(x^{(i)}, y^{(i)}) = i^{th}$ training example

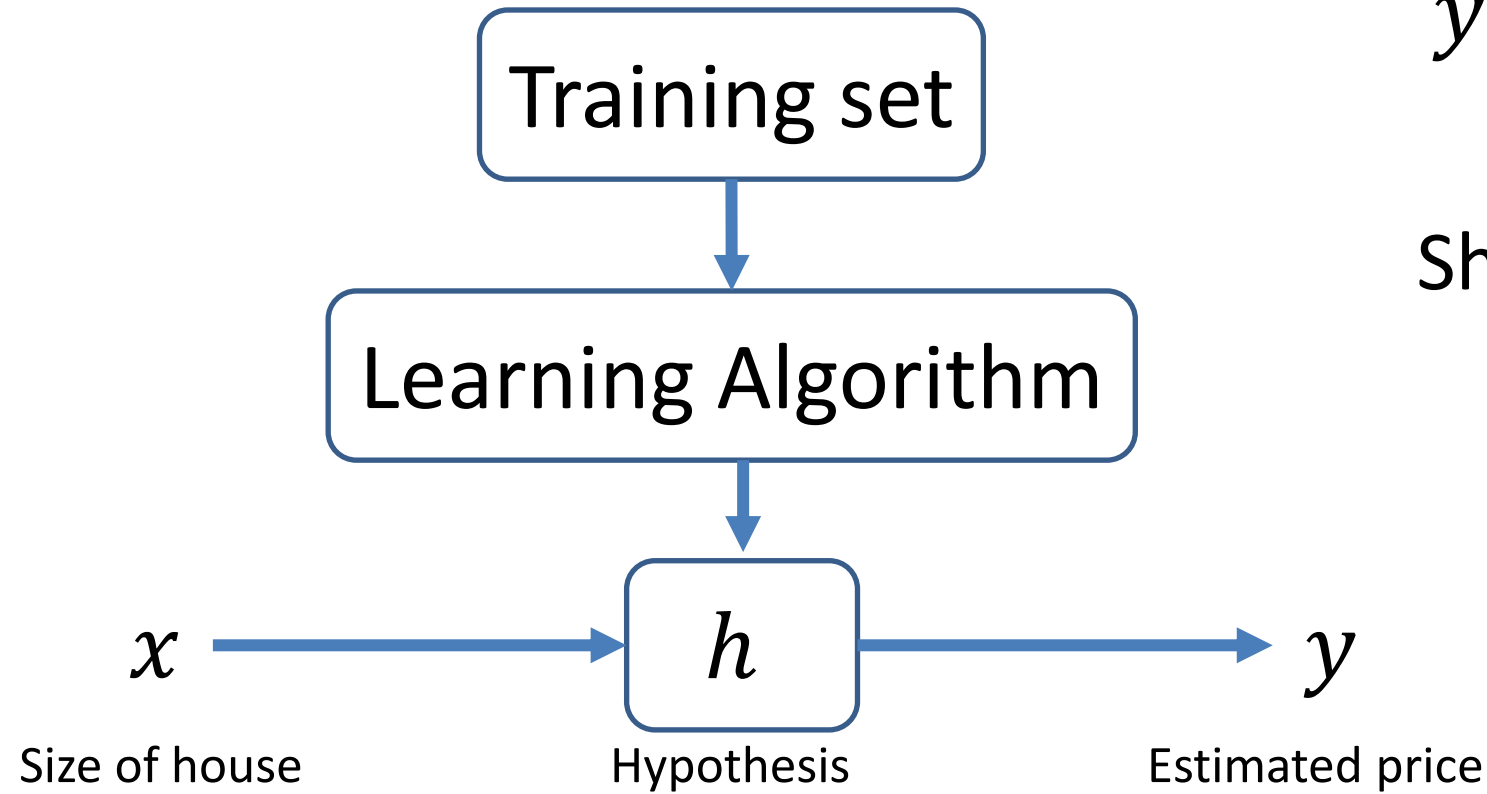
Examples:

$$x^{(1)} = 2104$$

$$x^{(2)} = 1416$$

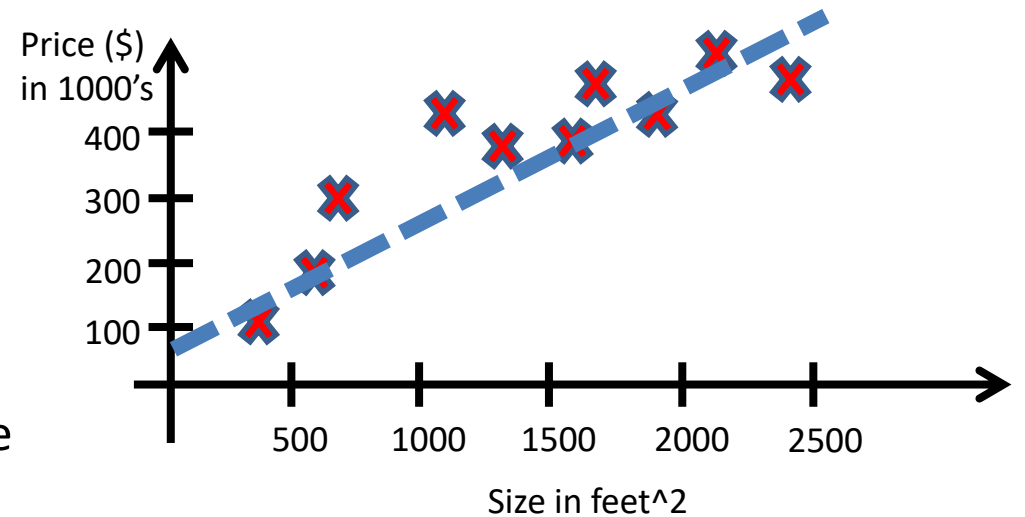
$$y^{(1)} = 460$$

Model representation



$$y = h_{\theta}(x) = \theta_0 + \theta_1 x$$

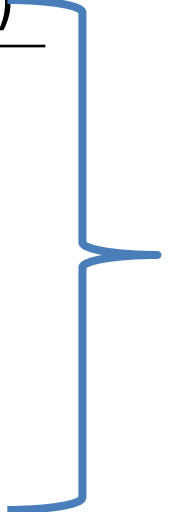
Shorthand $h(x)$



Univariate linear regression

Training set

Size in feet ² (x)	Price (\$) in 1000's (y)
2104	460
1416	232
1534	315
852	178
...	...



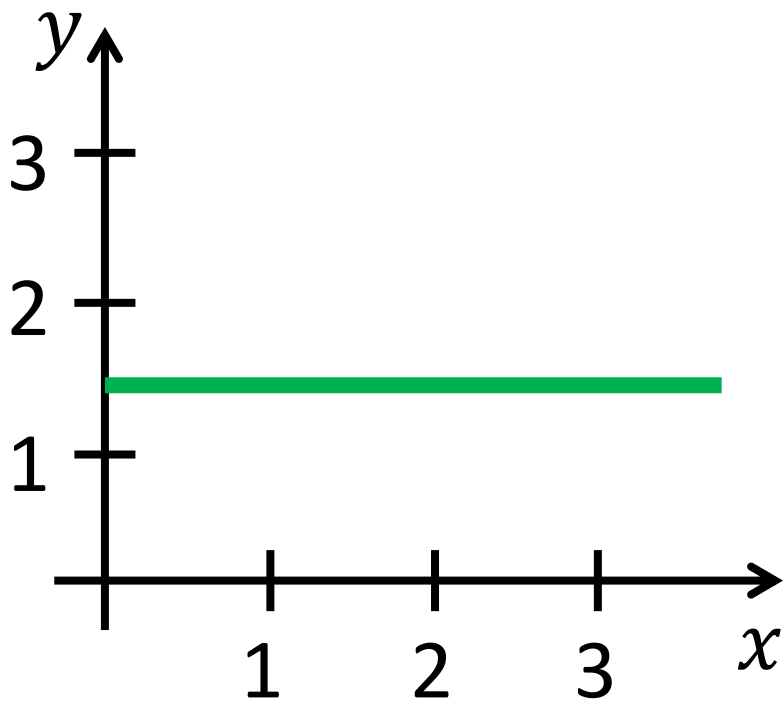
$m = 47$

- Hypothesis $h_{\theta}(x) = \theta_0 + \theta_1 x$

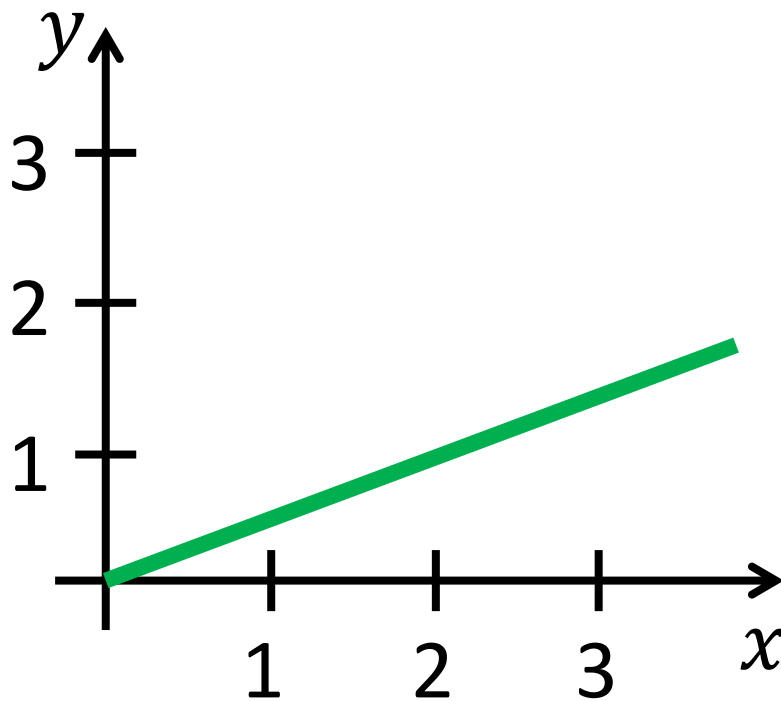
θ_0, θ_1 : parameters/weights

How to choose θ_i 's?

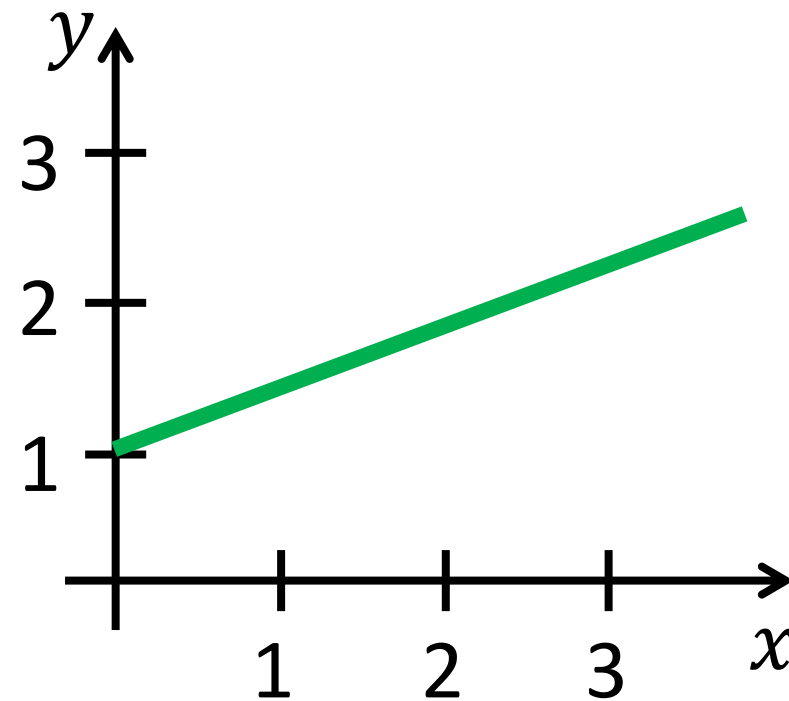
$$h_{\theta}(x) = \theta_0 + \theta_1 x$$



$$\begin{aligned}\theta_0 &= 1.5 \\ \theta_1 &= 0\end{aligned}$$



$$\begin{aligned}\theta_0 &= 0 \\ \theta_1 &= 0.5\end{aligned}$$



$$\begin{aligned}\theta_0 &= 1 \\ \theta_1 &= 0.5\end{aligned}$$

Best Fit Line in Linear Regression

- The goal of linear regression is to find a straight line that minimizes the error (the difference) between the observed data points and the predicted values.
- This line helps us predict the dependent variable for new, unseen data.
- To calculate best-fit line linear regression uses a traditional slope-intercept form which is given below,
- $Y_i = \beta_0 + \beta_1 X_i$
- where Y_i = Dependent variable, β_0 = constant/Intercept, β_1 = Slope/Intercept, X_i = Independent variable.
- This algorithm explains the linear relationship between the dependent(output) variable y and the independent(predictor) variable X using a straight line.

Types of Linear Regression

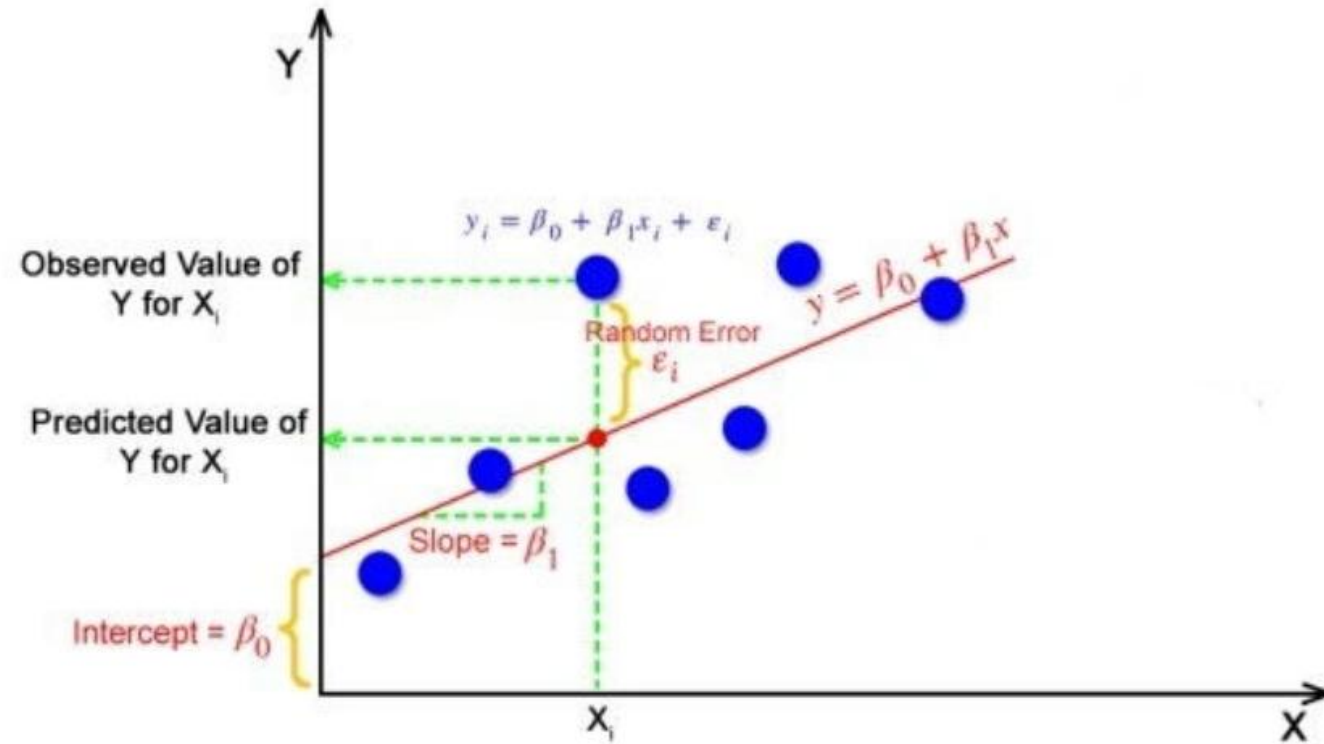
- **Simple Linear Regression** : is used when we want to predict a target value (dependent variable) using only one input feature (independent variable). It assumes a straight-line relationship between the two.
- **Formula:** $\hat{y} = \theta_0 + \theta_1 x$
- **Where:**
 - $\hat{y}$ is the predicted value
 - x is the input (independent variable)
 - θ_0 is the intercept (value of $\hat{y}$ when $x=0$)
 - θ_1 is the slope or coefficient (how much $\hat{y}$ changes with one unit of x)
- **Example:**
- Predicting a person's salary (y) based on their years of experience (x).

Types of Linear Regression

- **Multiple Linear Regression** involves more than one independent variable and one dependent variable.
- The equation for multiple linear regression is: $\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$
- **where:**
 - $\hat{y}$ is the predicted value
 - $x_1, x_2, \dots, x_n$ are the independent variables
 - $\theta_1, \theta_2, \dots, \theta_n$ are the coefficients (weights) corresponding to each predictor.
 - θ_0 is the intercept.
- The goal of the algorithm is to find the best Fit Line equation that can predict the values based on the independent variables.

But how the linear regression finds out which is the best fit line?

- The goal of the linear regression algorithm is to get the **best values for β_0 and β_1** to find the best fit line. The best fit line is a line that has the least error which means the error between predicted values and actual values should be minimum.
- In regression, the difference between the observed value of the dependent variable(y_i) and the predicted value(predicted) is called the residuals.
- $\epsilon_i = y_{predicted} - y_i$
- Where $y_{predicted} = \beta_0 + \beta_1 X_i$



Equation of the Best-Fit Line

- For simple linear regression (with one independent variable), the best-fit line is represented by the equation
- $y = mx + b$
- **Where:**
- **y** is the predicted value (dependent variable)
- **x** is the input (independent variable)
- **m** is the slope of the line (how much y changes when x changes)
- **b** is the intercept (the value of y when $x = 0$)
- The best-fit line will be the one that optimizes the values of m (slope) and b (intercept) so that the predicted y values are as close as possible to the actual data points.

Interpretation of the Best-Fit Line

- **Slope (m):** The slope of the best-fit line indicates how much the dependent variable (y) changes with each unit change in the independent variable (x). For example if the slope is 5, it means that for every 1-unit increase in x , the value of y increases by 5 units.
- **Intercept (b):** The intercept represents the predicted value of y when $x = 0$. It's the point where the line crosses the y -axis.
- In linear regression some hypothesis are made to ensure reliability of the model's results.

What is slope in a linear equation?

- The **slope** measures how steep a line is on a graph. It tells us **how much the (y)-value changes for every 1 unit change in (x)**.
- In the slope-intercept form of a line:
 $y = mx + b$ the slope is the number (m).
- Geometrically, slope is calculated as:
 $\left[\text{slope} = \frac{y_2 - y_1}{x_2 - x_1} \right]$ where $((x_1, y_1))$ and $((x_2, y_2))$ are two points on the line.
- The **intercept** b is the value of y when $x = 0$.

What is slope in a linear equation?

- **From an equation**

Suppose we have:

$$y = 3x + 2$$

Here, the slope is **3**, because the equation is already in $(y = mx + b)$ form.

- **y-intercept:** set $x = 0$

- $f(0) = 3(0) + 2 = 2$

What is slope in a linear equation?

- **From two points**

Find the slope of the line passing through $((2, 3))$ and $((5, 11))$.

- Using the formula:

$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{11 - 3}{5 - 2} = \frac{8}{3}$$

- We found slope $m = \frac{8}{3}$. Using point-slope form with $(2, 3)$:

- $y - 3 = \frac{8}{3}(x - 2)$

- Simplify:

- $y = \frac{8}{3}x - \frac{16}{3} + 3$

- $y = \frac{8}{3}x - \frac{7}{3}$

- Full function:

- $f(x) = \frac{8}{3}x - \frac{7}{3}$

What is slope in a linear equation?

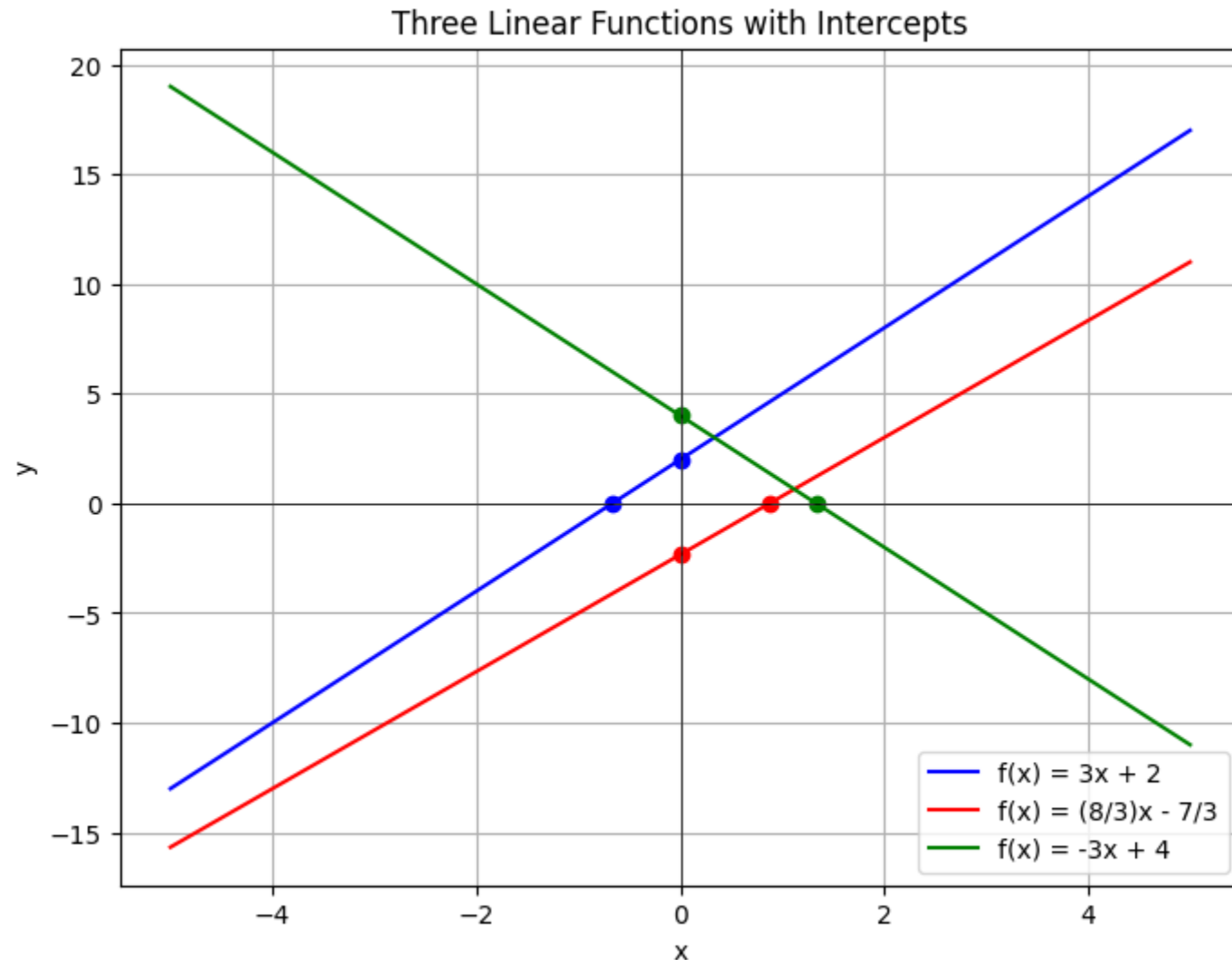
- **Negative slope**

Suppose the line passes through $((0, 4))$ and $((2, -2))$:

$$m = \frac{-2 - 4}{2 - 0} = \frac{-6}{2} = -3$$

- We found slope $m = -3$. Using point-slope form with $(0, 4)$:
- $y - 4 = -3(x - 0)$
- $y = -3x + 4$
- Full function:
- $f(x) = -3x + 4$

- A positive slope means the line rises from left to right,
- A negative slope means the line falls from left to right
- A slope of 0 means the line is horizontal,
- An undefined slope (division by 0) means the line is vertical.

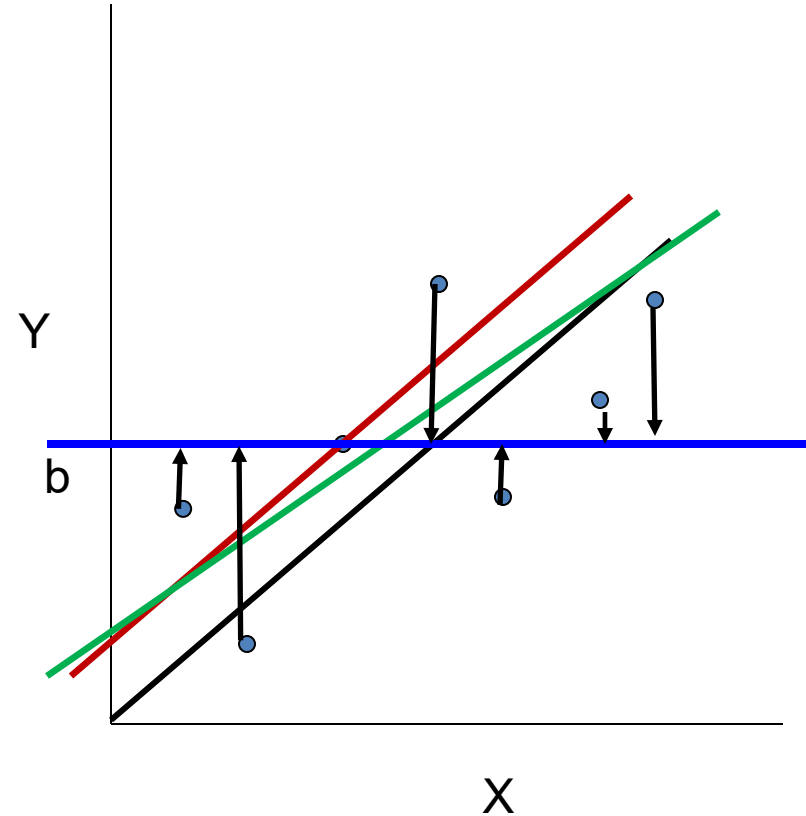


Minimizing the Error: The Least Squares Method

- To find the best-fit line, we use a method called Least Squares. The idea behind this method is to minimize the sum of squared differences between the actual values (data points) and the predicted values from the line. These differences are called residuals.
- The formula for residuals is:
- $Residual = y_i - \hat{y}_i$
- **Where:**
 - y_i is the actual observed value
 - $\hat{y}_i$ is the predicted value from the line for that x_i
- The least squares method minimizes the sum of the squared residuals:
- $Sum of squared errors (SSE) = \Sigma(y_i - \hat{y}_i)^2$
- This method ensures that the line best represents the data where the sum of the squared differences between the predicted values and actual values is as small as possible.

Best Fit (Least Squares)

- Assume you have been able to gather data (several points) where:
 - X is the number of rooms in a house.
 - Y is the cost of a house.
- The question is what is the best line that fits the data?
- Several options: each with a different intercept and a slope $Y = mx + b$
- How do we evaluate the goodness of each line? (how well this line fits data)
- Find the **sum of squared residuals** (difference between the real data and the line).
- The line with the least sum is best!!

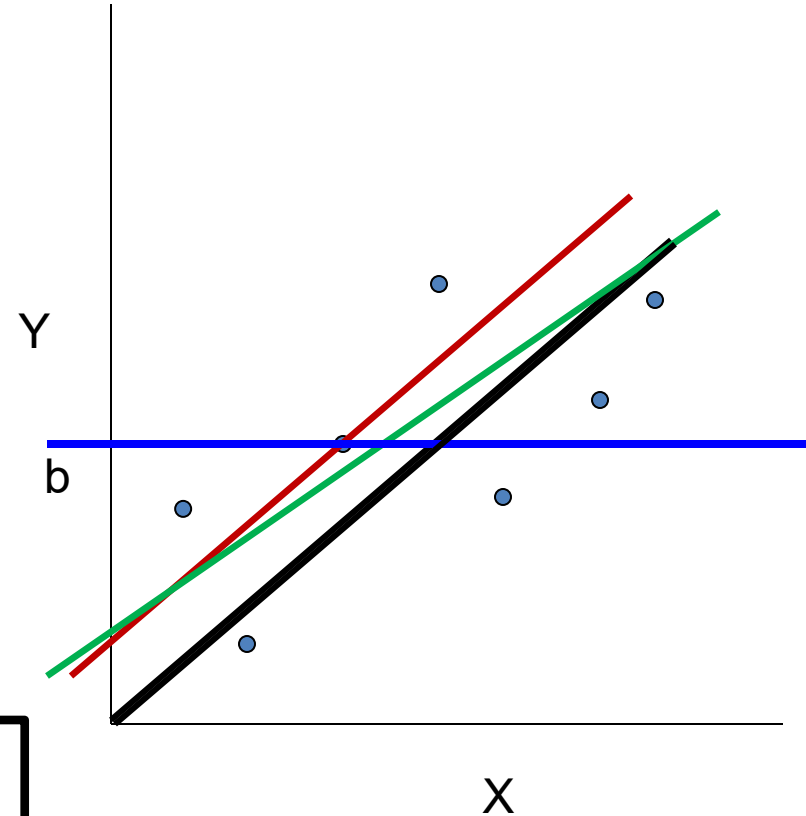


$$(b-y_1)^2 + (b-y_2)^2 + (b-y_3)^2 + (b-y_4)^2 + (b-y_5)^2 + (b-y_6)^2$$

→ Loss Function

Best Fit (Least Squares)

- Find the sum of squared residuals (difference between the real data and the line).
- The line with the least sum is best!!
- Sum of Squared Residuals:
 - Black Line: 13.2
 - Red Line: 17.8
 - Green Line: 15.3
 - Blue Line: 45.2
- Therefore, there is a **sweet spot** to find the appropriate line
- Objective is to find the best slope (m) and intercept (b) of $Y = mX + b$



$$((m \cdot x_1 + b) - y_1)^2 + ((m \cdot x_2 + b) - y_2)^2 + ((m \cdot x_3 + b) - y_3)^2 + ((m \cdot x_4 + b) - y_4)^2 + ((m \cdot x_5 + b) - y_5)^2 + \dots$$

Cost function for Linear Regression

- As we have discussed earlier about best fit line in linear regression, its not easy to get it easily in real life cases so we need to calculate errors that affects it.
- These errors need to be calculated to mitigate them. The difference between the predicted value $\hat{y}$ and the true value Y and it is called cost function or the loss function.

Cost function for Linear Regression

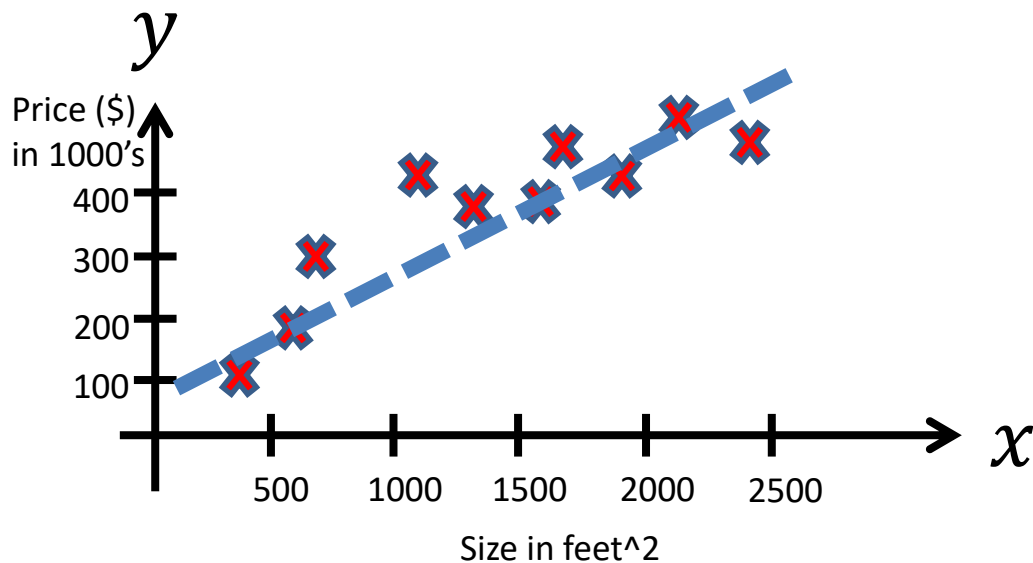
- In Linear Regression, the Mean Squared Error (MSE) cost function is employed, which calculates the average of the squared errors between the predicted values $\hat{y}_i$ and the actual values y_i . The purpose is to determine the optimal values for the intercept θ_1 and the coefficient of the input feature θ_2 providing the best-fit line for the given data points. The linear equation expressing this relationship is $\hat{y}_i = \theta_1 + \theta_2 x_i$.
- MSE function can be calculated as:
- Cost function $(J) = \frac{1}{n} \sum_n^i (\hat{y}_i - y_i)^2$

Cost function for Linear Regression

- Utilizing the MSE function, the iterative process of gradient descent is applied to update the values of θ_1 and θ_2 . This ensures that the MSE value converges to the global minima, signifying the most accurate fit of the linear regression line to the dataset.
- This process involves continuously adjusting the parameters θ_1 and θ_2 based on the gradients calculated from the MSE.
- The final result is a linear regression line that minimizes the overall squared differences between the predicted and actual values, providing an optimal representation of the underlying relationship in the data.

Cost function

- Idea:
Choose θ_0, θ_1 so that $h_\theta(x)$ is close to y for our training example (x, y)



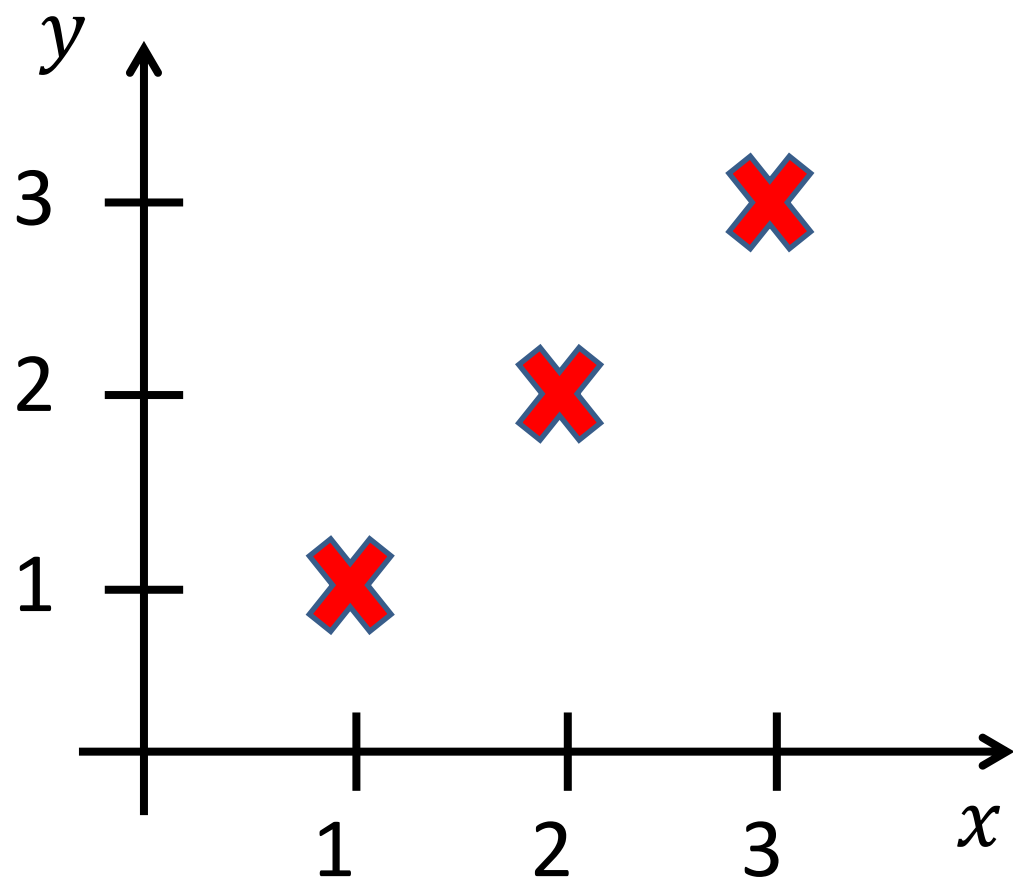
$$\underset{\theta_0, \theta_1}{\text{minimize}} \quad \frac{1}{m} \sum_{i=1}^m (h_\theta(x^{(i)}) - y^{(i)})^2$$

$$h_\theta(x^{(i)}) = \theta_0 + \theta_1 x^{(i)}$$

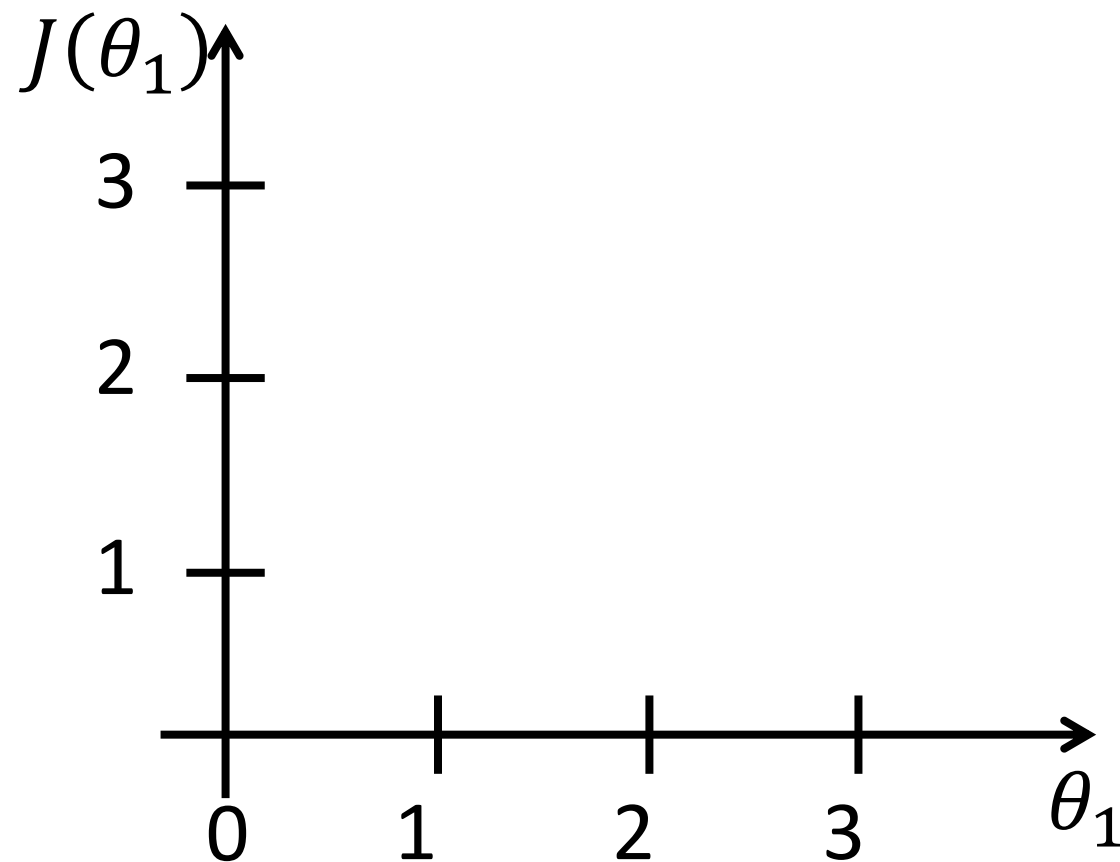
$$J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^m (h_\theta(x^{(i)}) - y^{(i)})^2$$

$$\underset{\theta_0, \theta_1}{\text{minimize}} \quad \boxed{J(\theta_0, \theta_1)} \quad \text{Cost function}$$

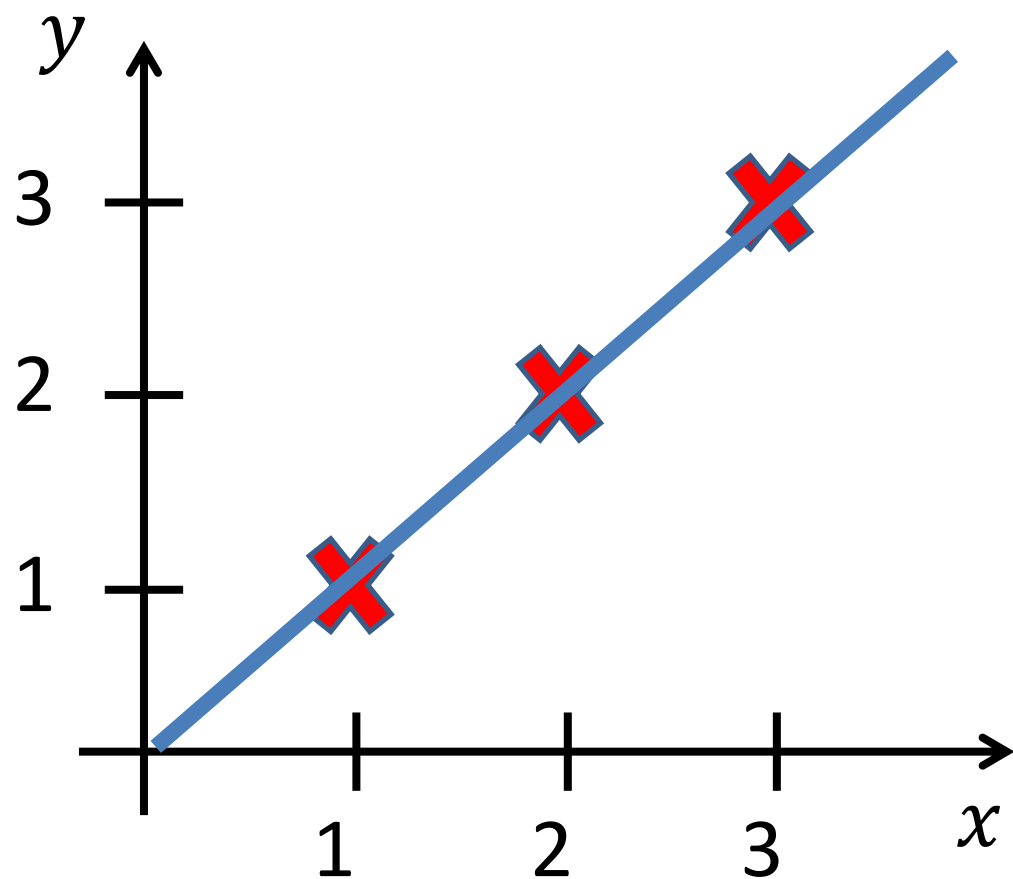
$h_{\theta}(x)$, function of x



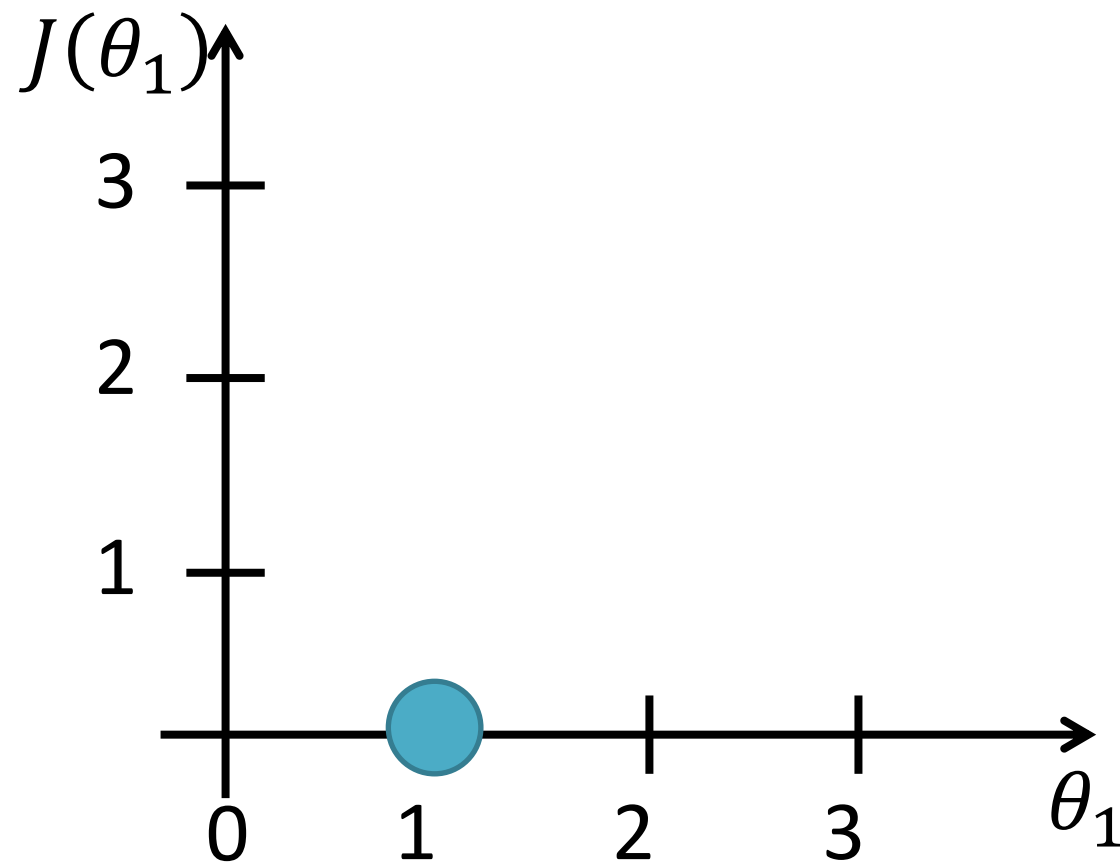
$J(\theta_1)$, function of θ_1



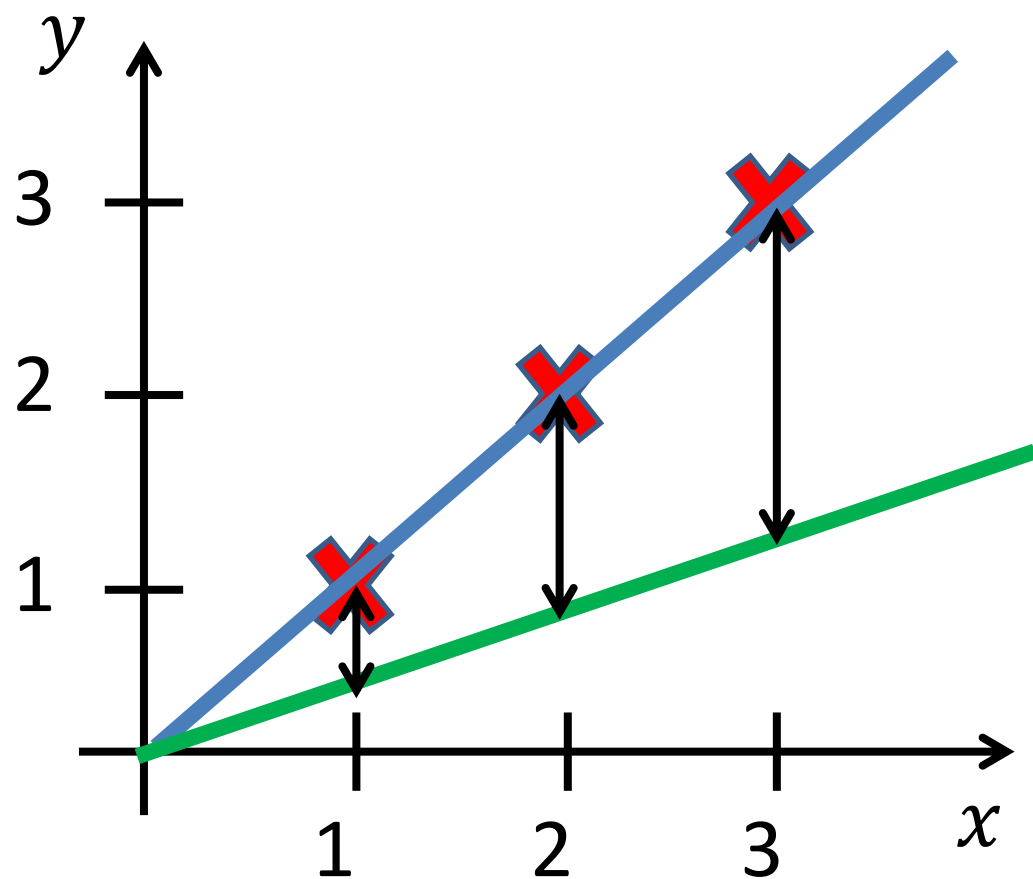
$h_{\theta}(x)$, function of x



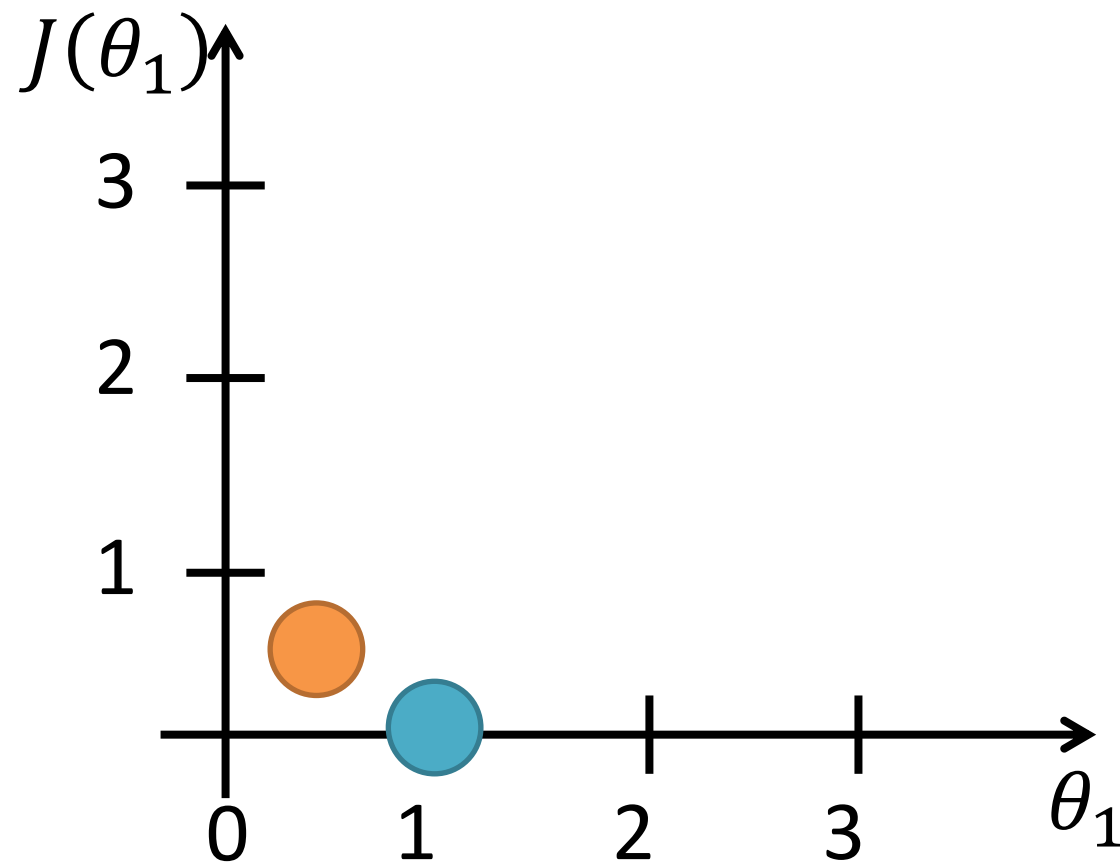
$J(\theta_1)$, function of θ_1



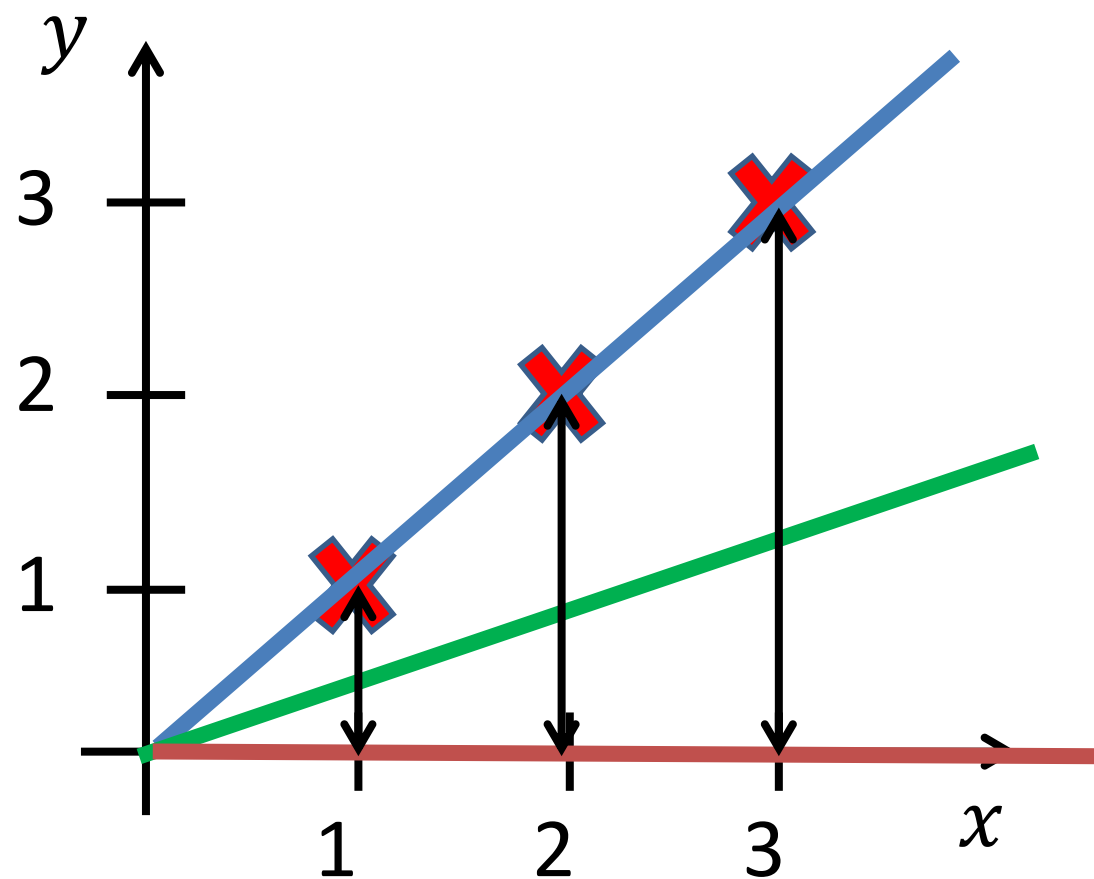
$h_{\theta}(x)$, function of x



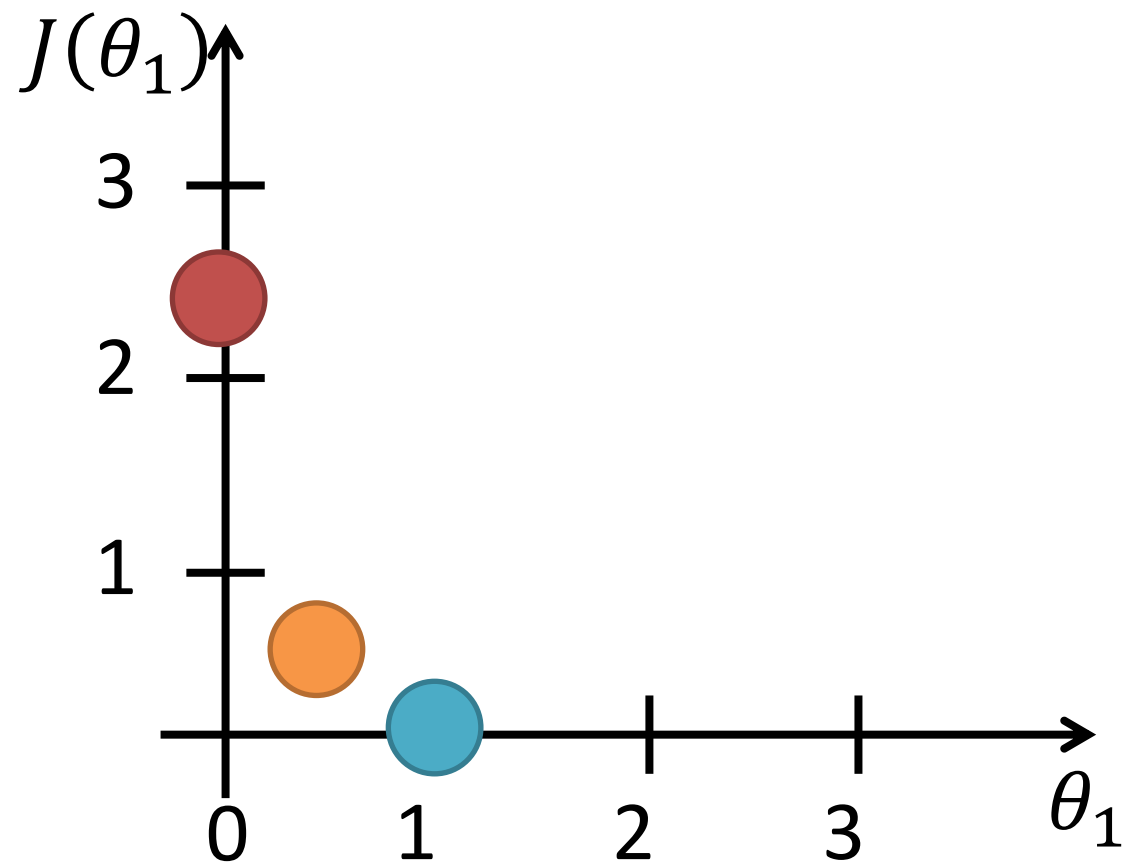
$J(\theta_1)$, function of θ_1



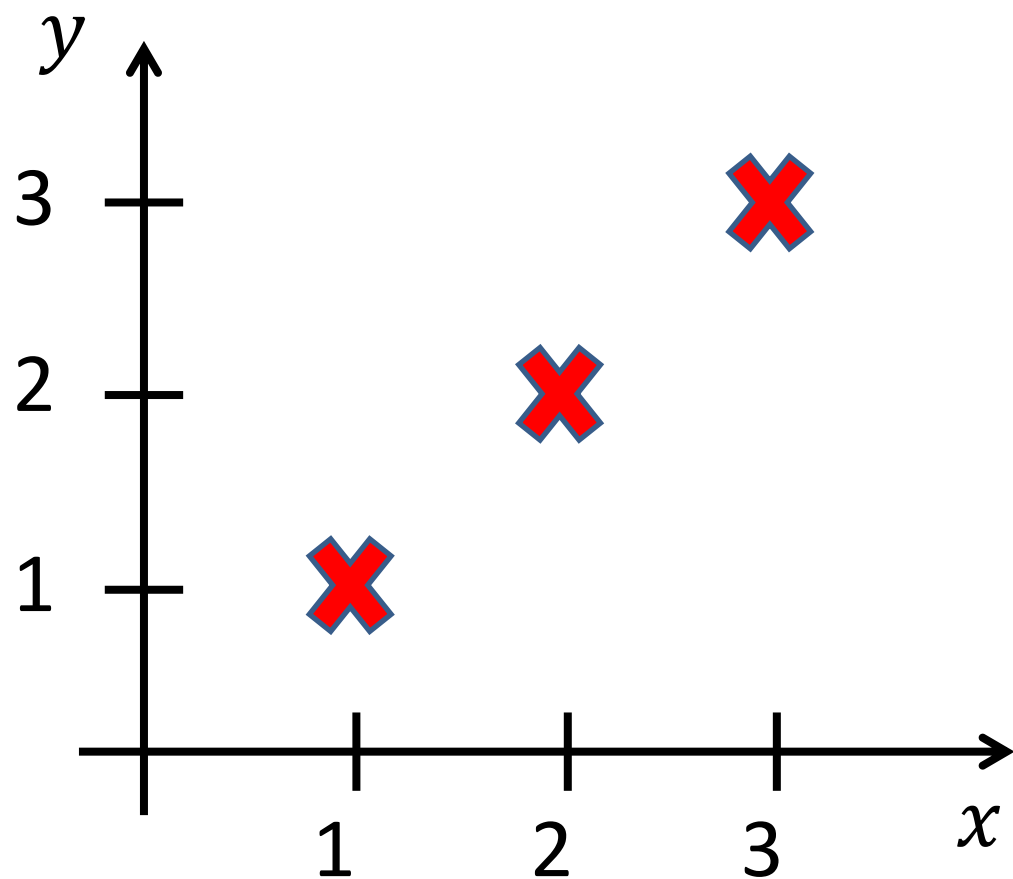
$h_{\theta}(x)$, function of x



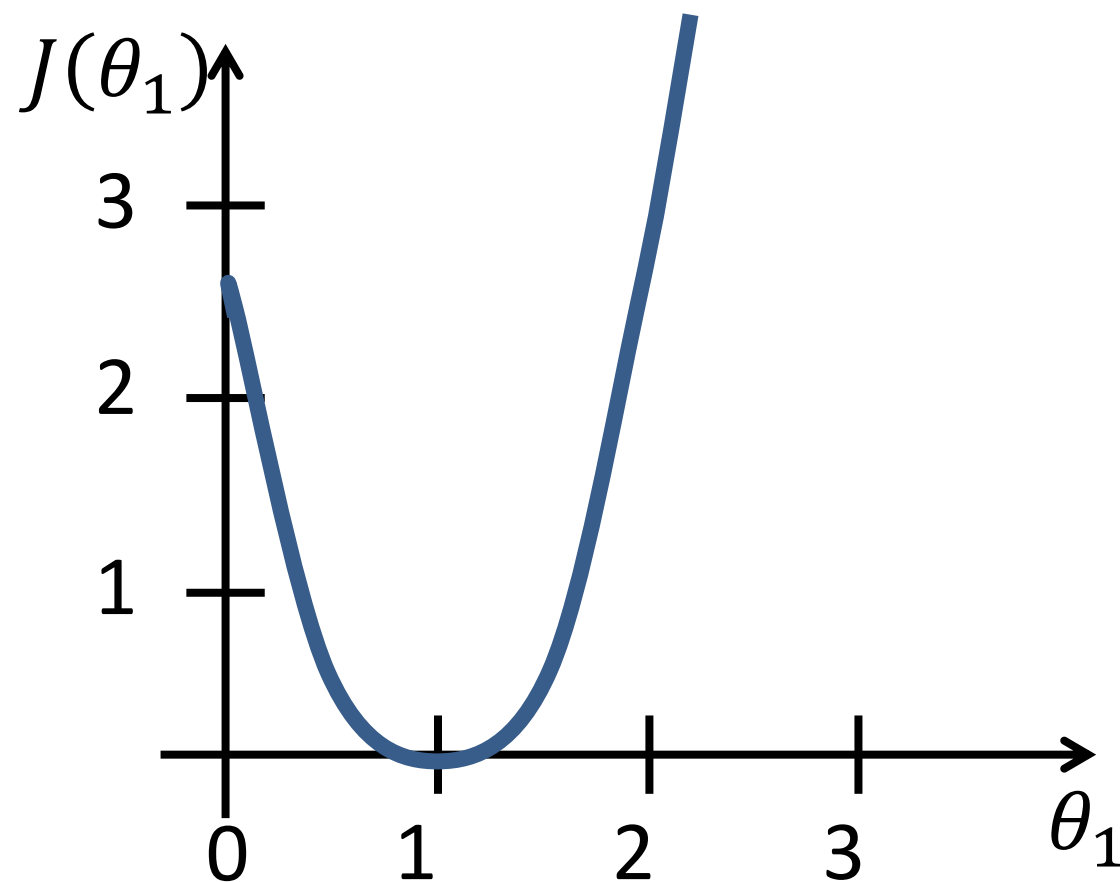
$J(\theta_1)$, function of θ_1



$h_{\theta}(x)$, function of x

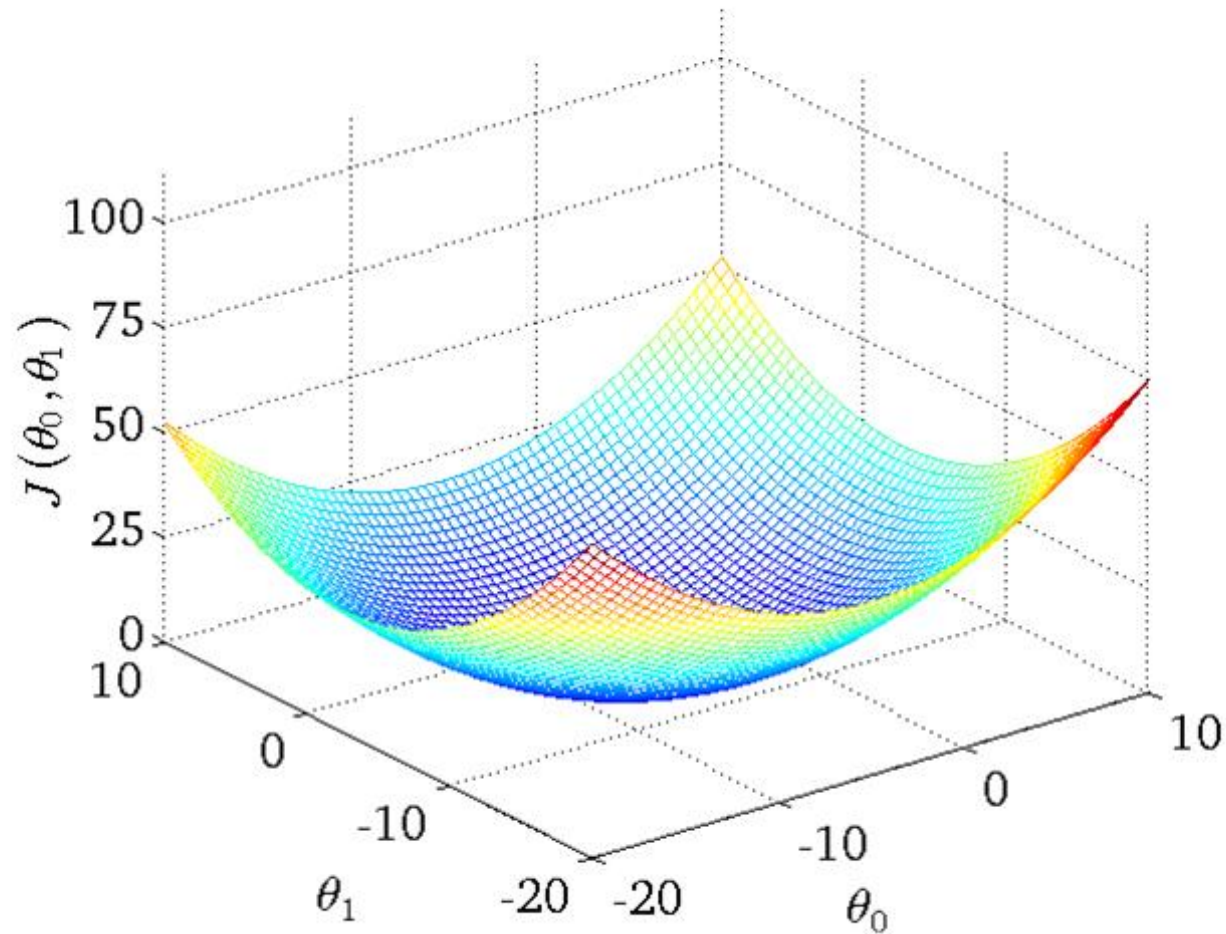


$J(\theta_1)$, function of θ_1



- **Hypothesis:** $h_{\theta}(x) = \theta_0 + \theta_1 x$
- **Parameters:** θ_0, θ_1
- **Cost function:** $J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$
- **Goal:** minimize $J(\theta_0, \theta_1)$
 θ_0, θ_1

Cost function

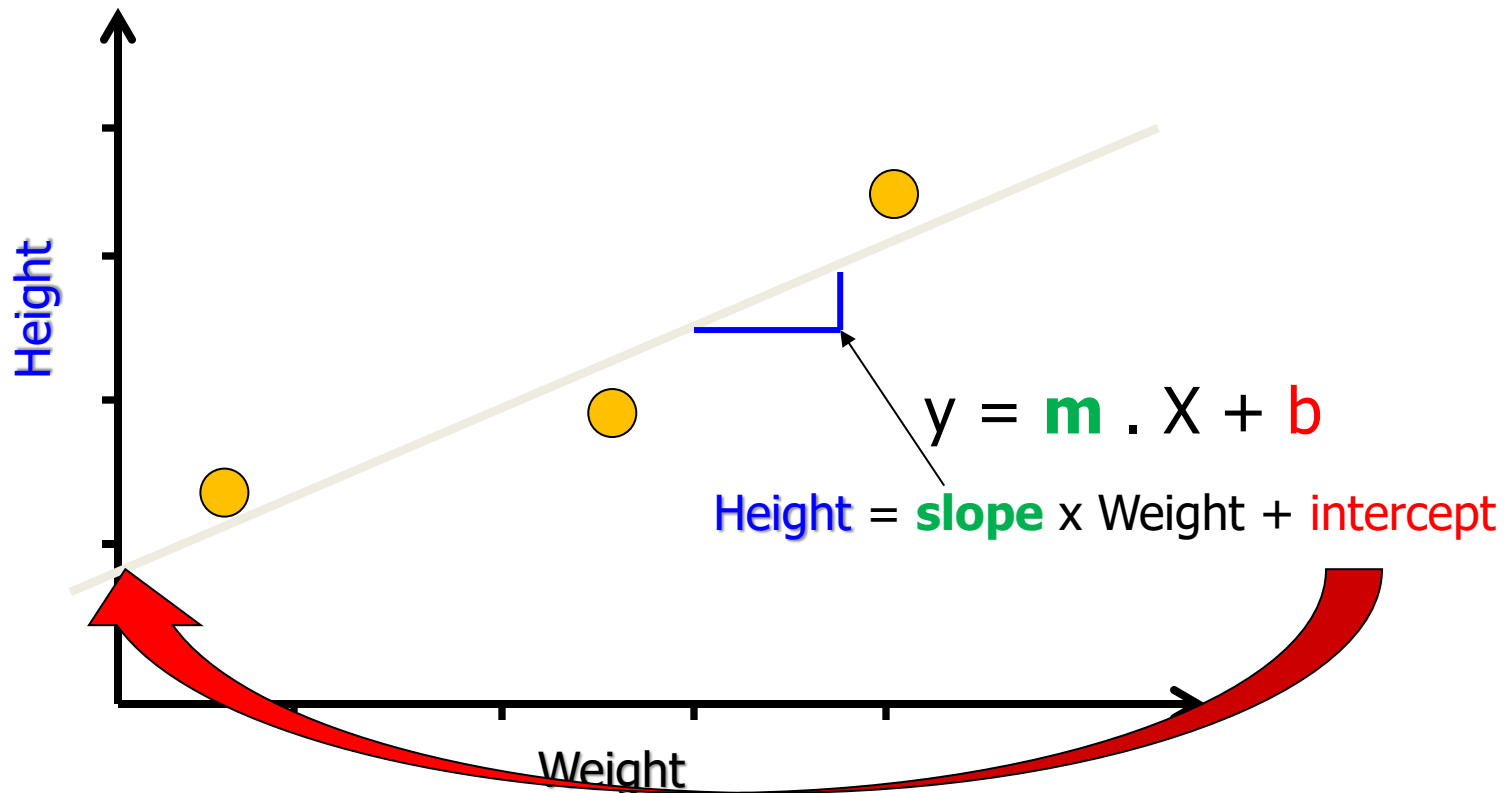


Cost function

- Now we have calculated loss function we need to optimize model to mitigate this error and it is done through gradient descent.

Optimization

- In Machine Learning (Classification/Regression) we tend to use *optimization* to train models (ANNs, Regression, SVM, DNNs).
- Linear Regression, Least-Squares Regression → line with best fit
- When we fit a line with Linear Regression, we optimize the Intercept and Slope (i.e., weights of the features)

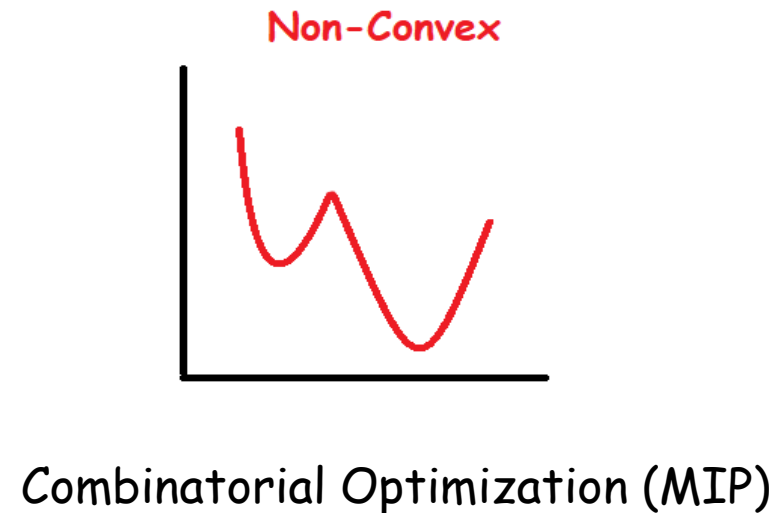
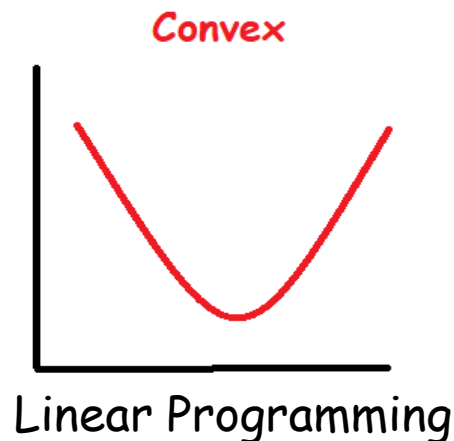


Optimization

- Several machine learning algorithms such as ANNs, SVM, Linear Regression use Gradient Descent as a form of
- Optimization technique to minimize the loss function (objective function) which represent the error
- Linear regression/Least Square Regression attempts to **fit the best line** among the data points
- Fitting the best line translates to finding the **best slope and intercept**

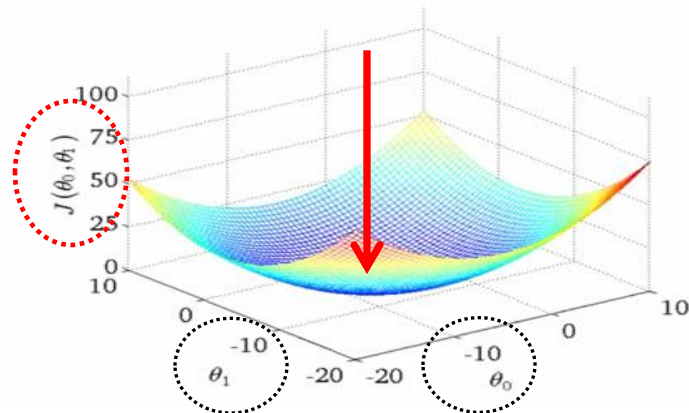
ML & Optimization

- Optimization in Machine Learning refers to minimizing a cost function $J(w)$ parameterized by the predictive model's parameters.
- The **Cost Function** $\rightarrow (y - y')^2$ or $|y - y'|$... (i.e., difference between the actual label and the **prediction** made by the ML model)
- In order to minimize the cost function, we aim to find the global minimum which is quite feasible if the objective function is convex.
- Nonetheless, in many scenarios such as deep learning task our objective function tends to be non-convex, thereby finding the lowest possible value of the objective function has been highly regarded as a suitable solution.



Loss (Cost) Function

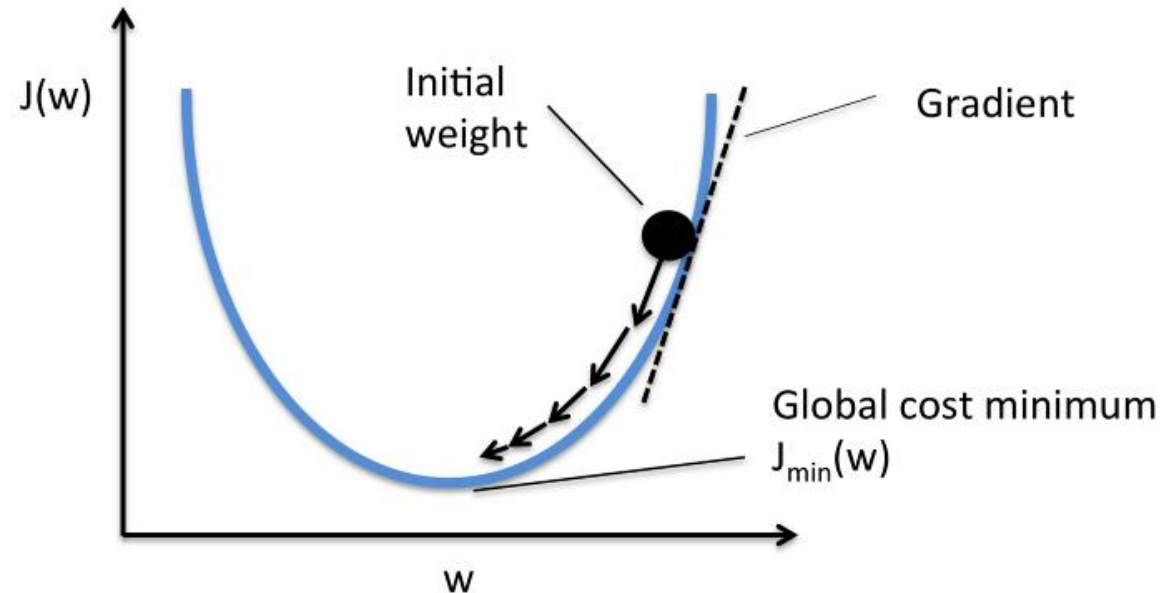
- The function we want to minimize or maximize is called the objective function or criterion.
- When we are minimizing it, we may also call it the (a) *cost function*, (b) *loss function*, or (c) *error function*.
- The x and y axes represent the values of the weights/parameters that can be adjusted to minimize the value of the *loss function*.
- The z axis represents the value of the *loss function* for a particular value of two weights/parameters.
- Our goal is to find the particular value of weights for which the loss is minimum. Such a point is called a minima for the *loss function*.



How do we know we reached
Local minima?

Gradient: Idea

- To find the local minimum of the function we take steps proportional to the negative of the gradient of the *cost function* $J(w)$ at the current point.
 - The gradient indicates the direction of greatest change of a function.
 - The gradient gives a vector that points in the direction of steepest ascent
- We start with a random point on the *objective function* and move in the negative direction towards the global/local minimum.

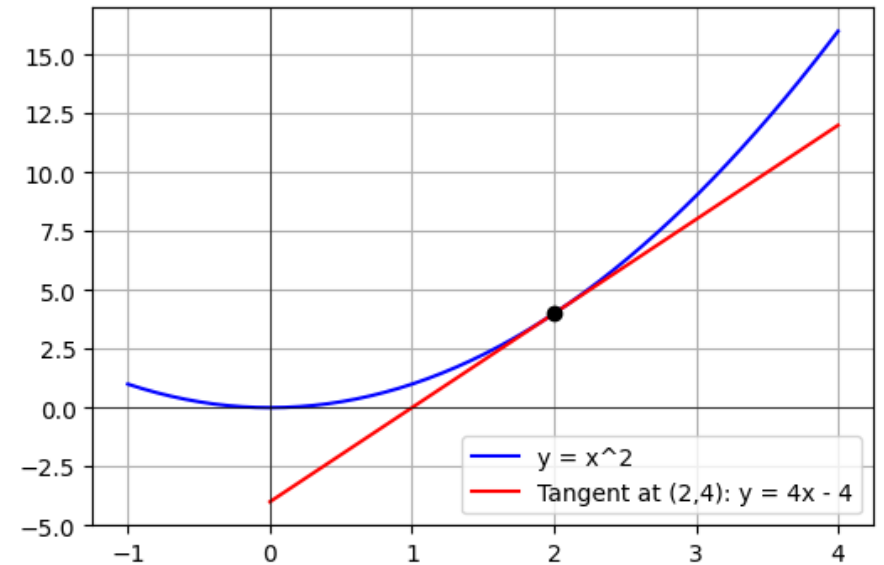


Gradient

- **Gradient of a line (simple slope example)**
- Equation:
- $y = 3x + 2$
- Here the **gradient = 3**.
- It means: for every +1 increase in x , y goes up by +3.
- Example:
 - If $x = 1$, $y = 5$.
 - If $x = 2$, $y = 8$.
 - Change in $y = 3$, change in $x = 1$.
 - So gradient = **3**.

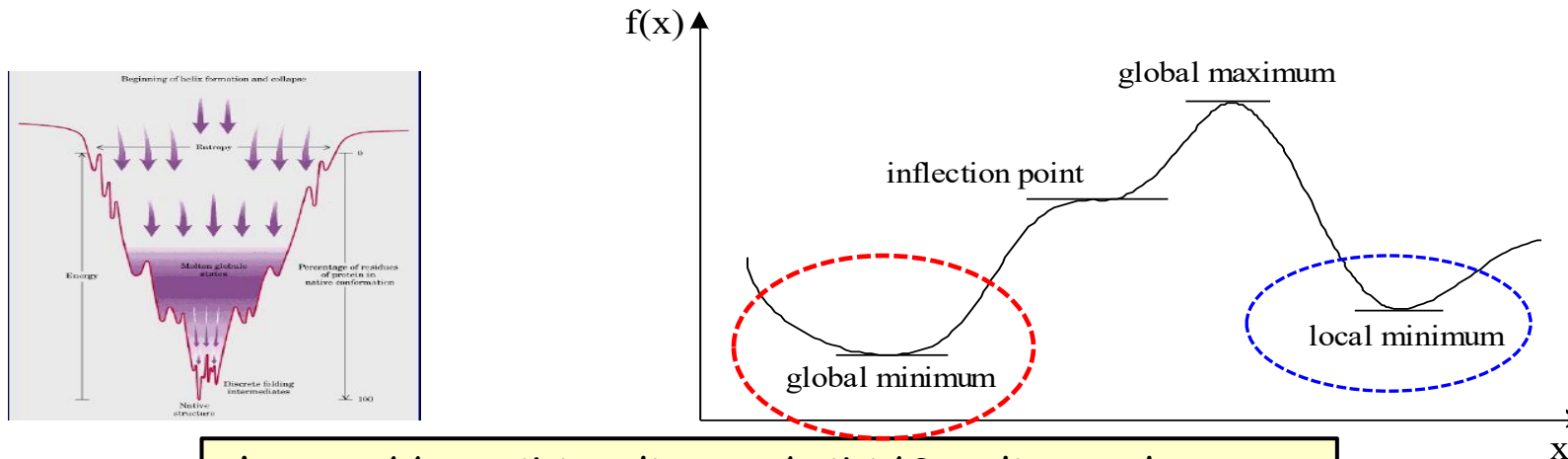
Gradient

- **Gradient of a curve (derivative example)**
- Function:
- $f(x) = x^2$
- Derivative (gradient function):
- $f'(x) = 2x$
- 👉 At $x = 2$:
- $f'(2) = 2(2) = 4$
- So the slope (gradient) of the tangent at $(2, 4)$ is **4**.
If you move 1 step in x , the curve rises about 4 steps in y .



Gradient Descent (or Ascent)

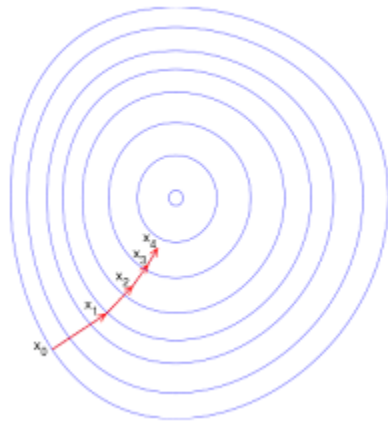
- Gradient of a function indicates the direction of greatest change.
- Optimization is finding the "best" value of a function which is the minimum value of the function.
- The number of "turning points" of a function depend on the order of the function.
- Not all turning points are minima.
- Every minimum is a "**local**" minimum.
- The least of all the minimum points is called the "global" minimum.



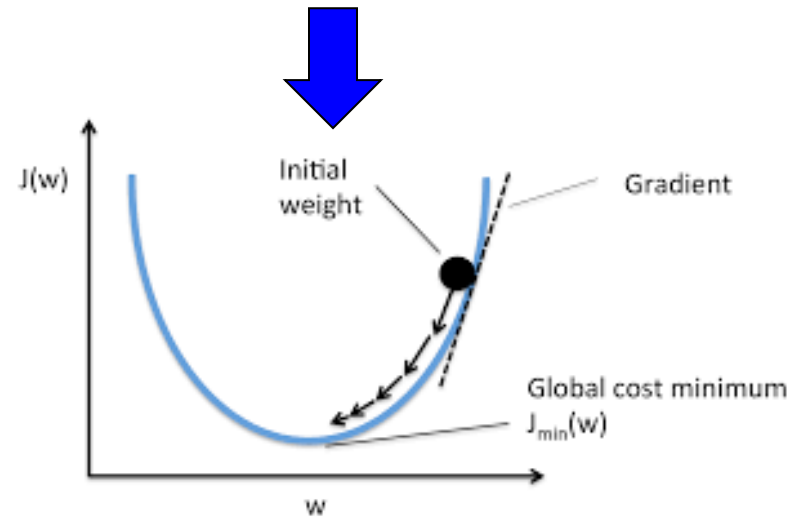
https://en.wikipedia.org/wiki/Gradient_descent

What is Gradient Descent?

- Gradient Descent is a first-order iterative optimization algorithm for finding a local min of a differentiable function.
- Gradient Descent is used in machine learning (ANNs, LR, SVM, ..) to find the best values of a function's parameters that minimize a cost function as far as possible.
- To find a local minimum of a function using gradient descent, we take steps proportional to the negative of the gradient of the function at the current point.



Contour



Gradient Descent (or Ascent)

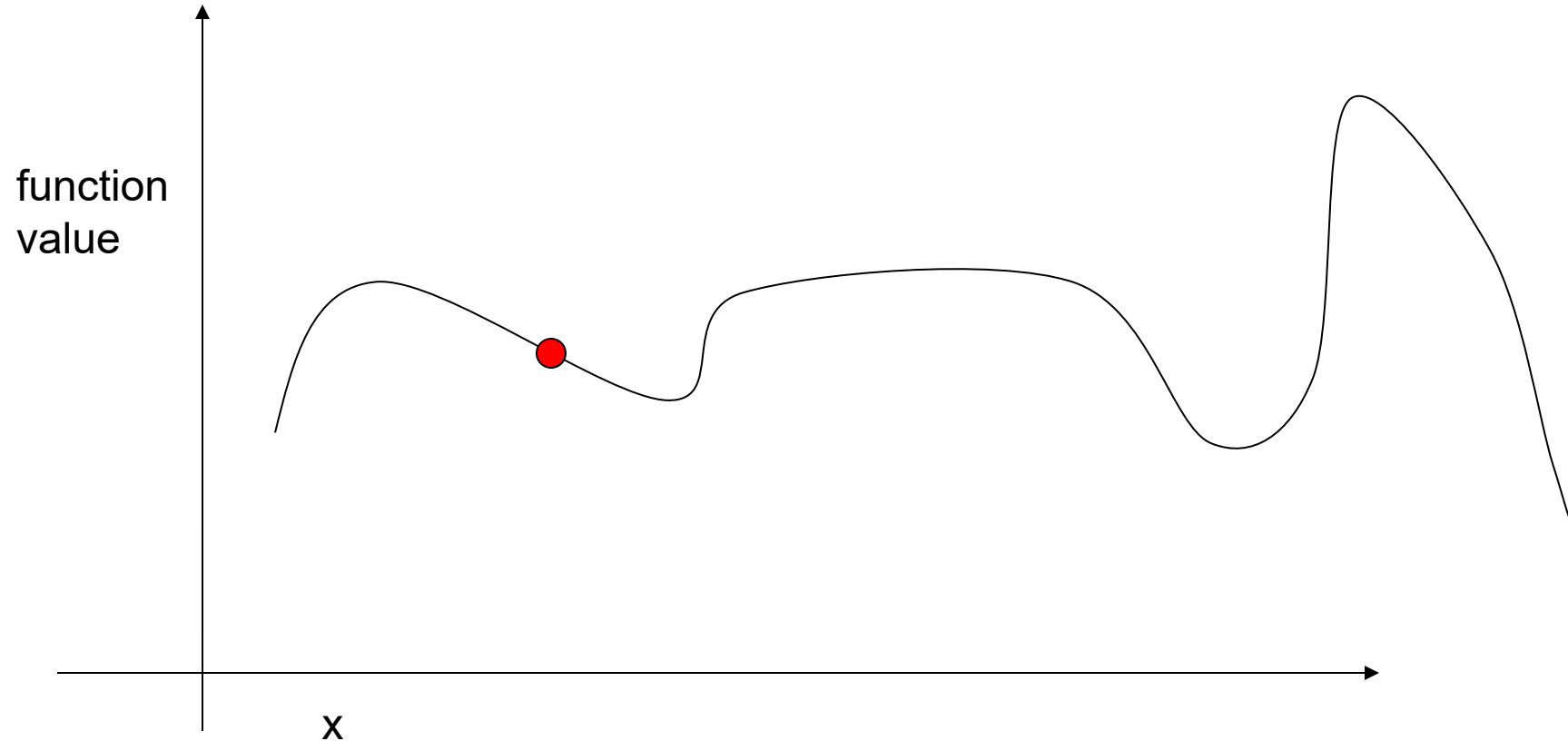
- Simple modification to Hill Climbing
 - Generally, assumes a continuous state space
- Idea is to take more intelligent steps
- Look at local gradient: the direction of largest change
- Take step in that direction
 - Step size should be proportional to gradient
- Tends to yield much faster convergence to maximum

If we have multiple minima can GD escape local minima?

Based on current description of GD, it will get stuck in local minima

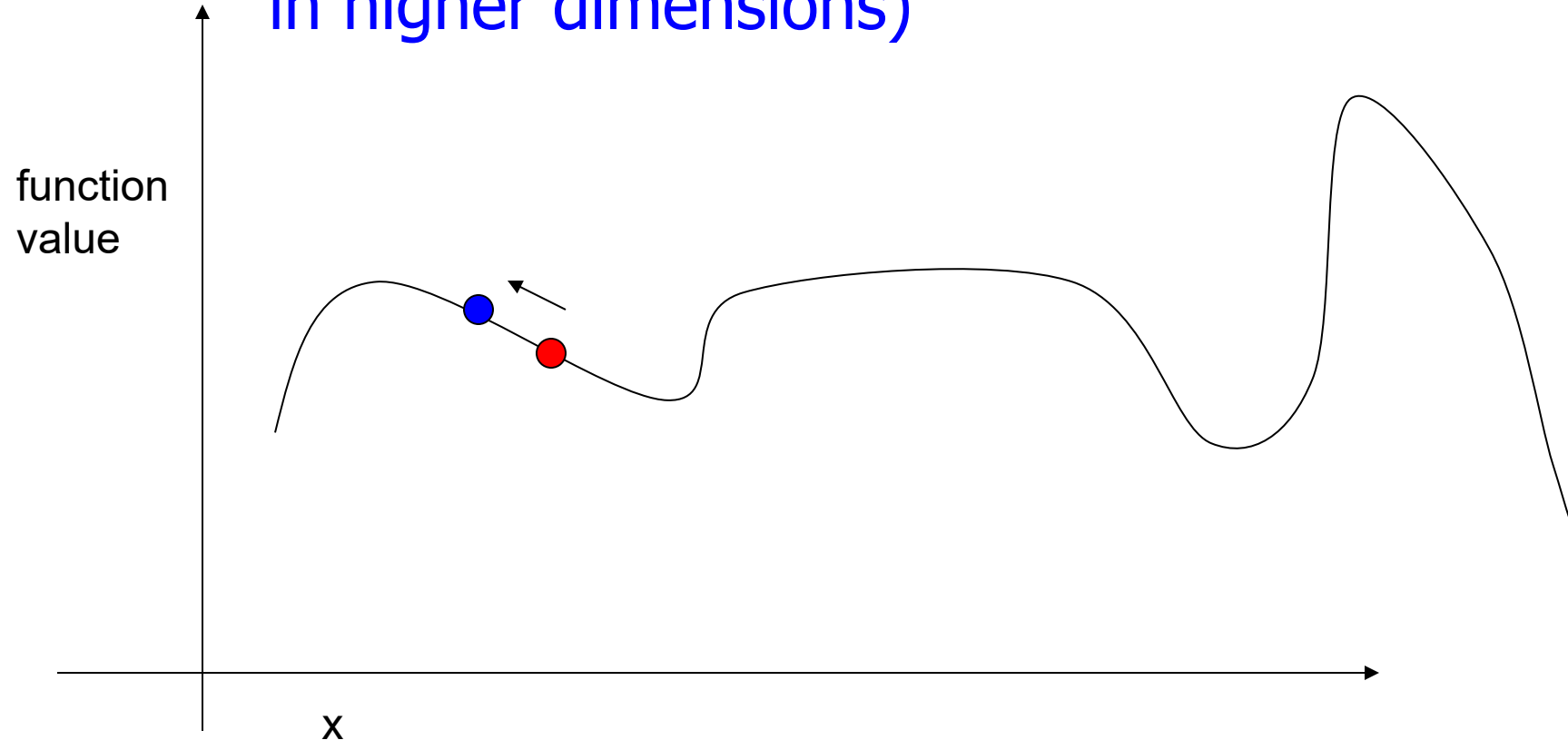
Gradient Ascent

- Random Starting Point



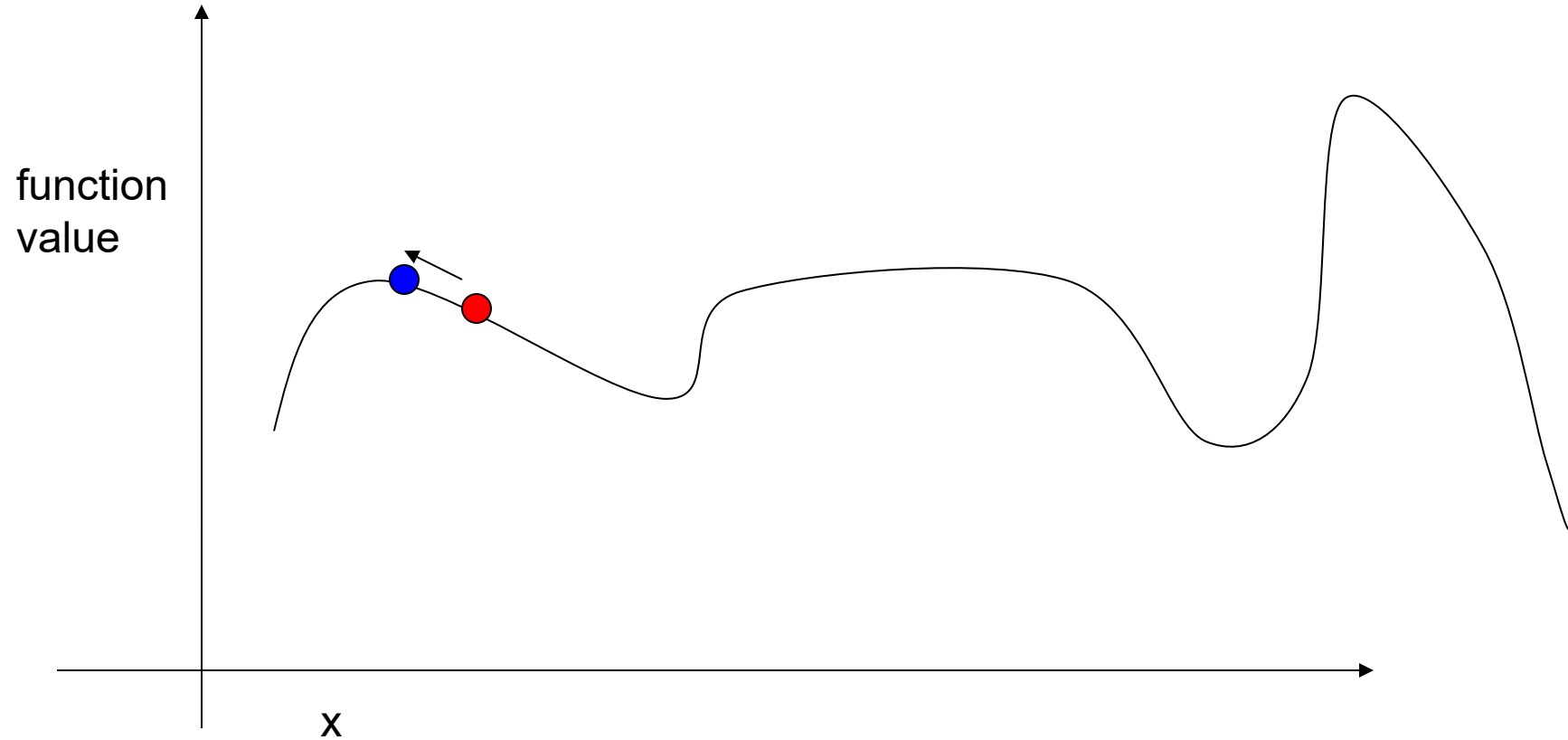
Gradient Ascent

- Take step in direction of largest increase
(obvious in 1D, must be computed
in higher dimensions)



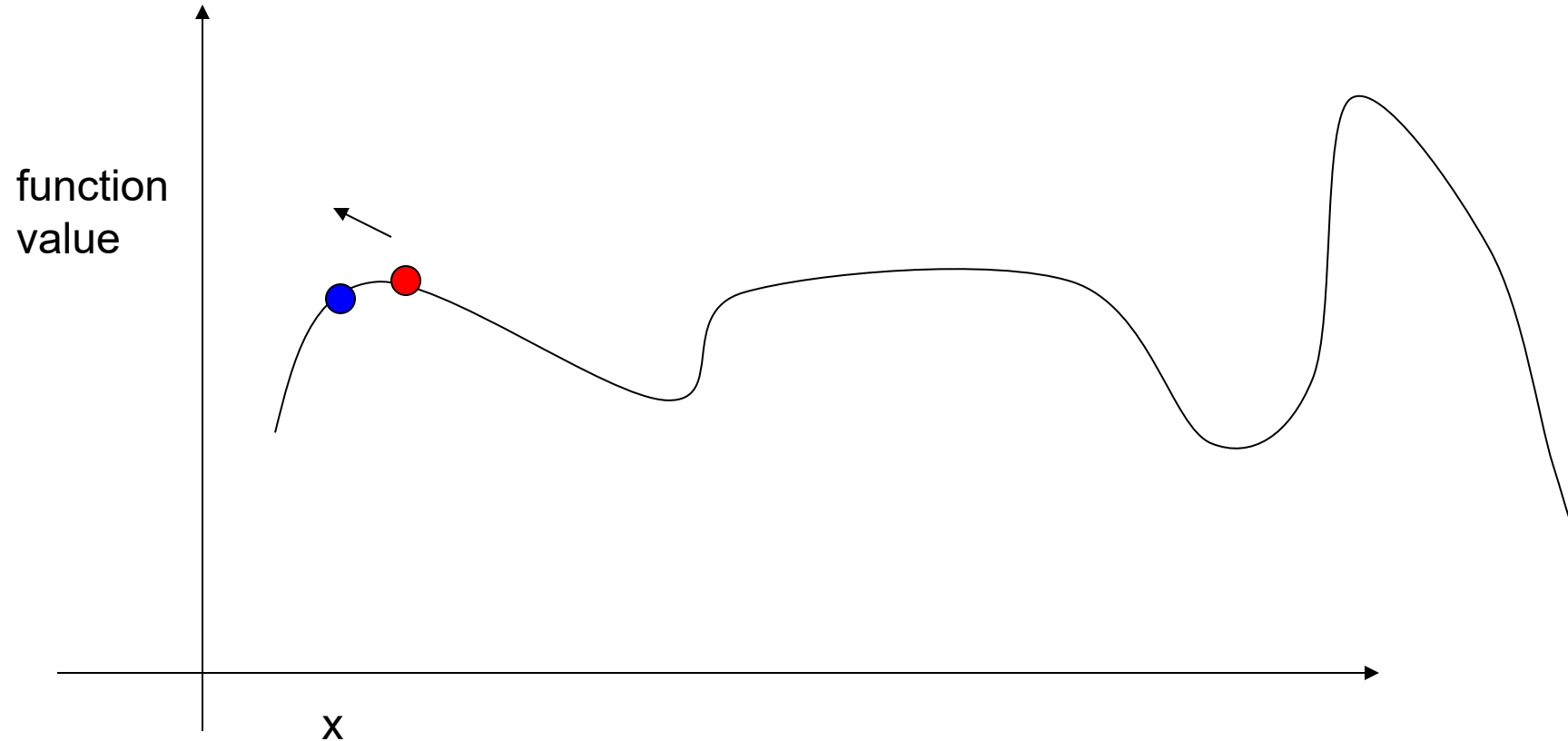
Gradient Ascent

- Repeat



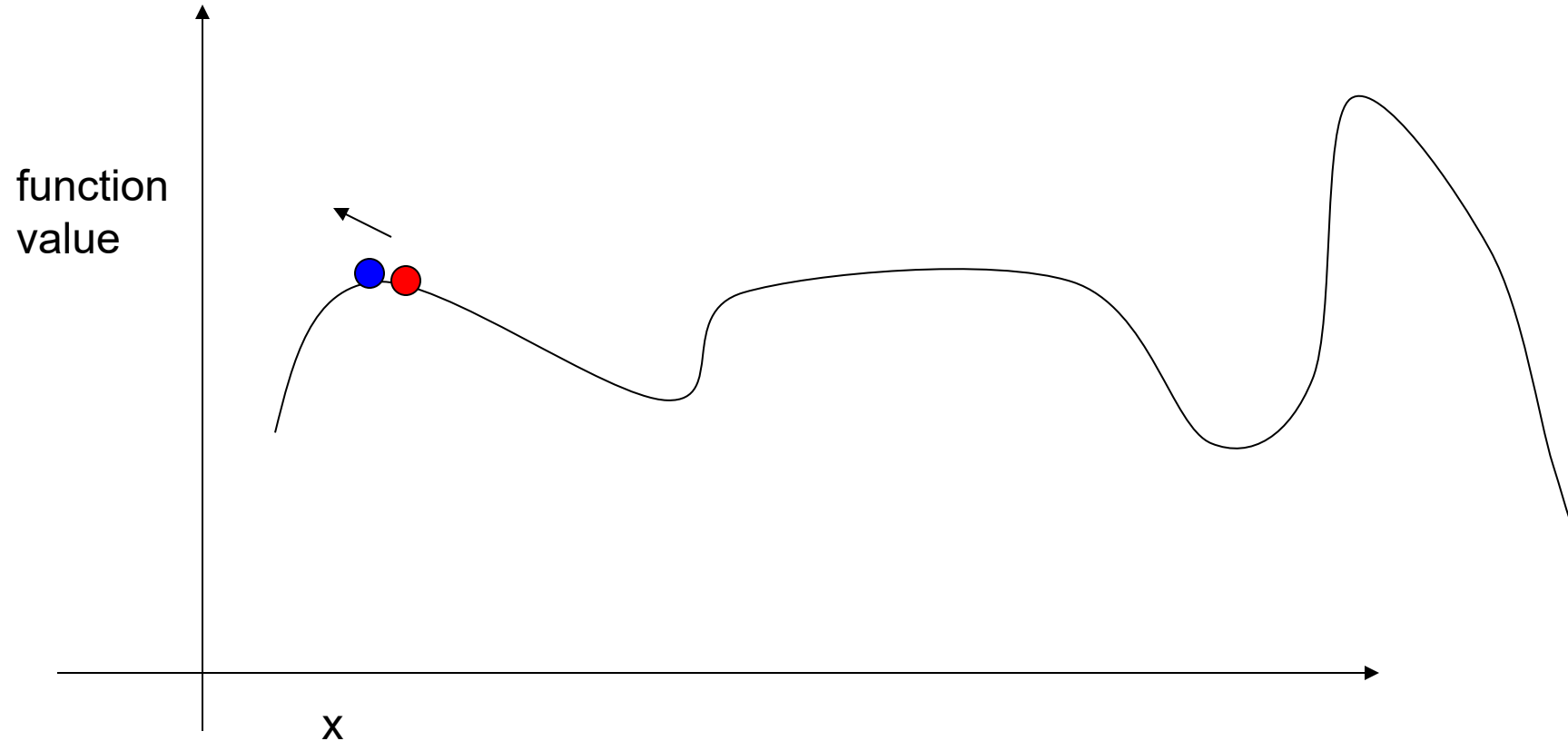
Gradient Ascent

- Next step is actually lower, so stop



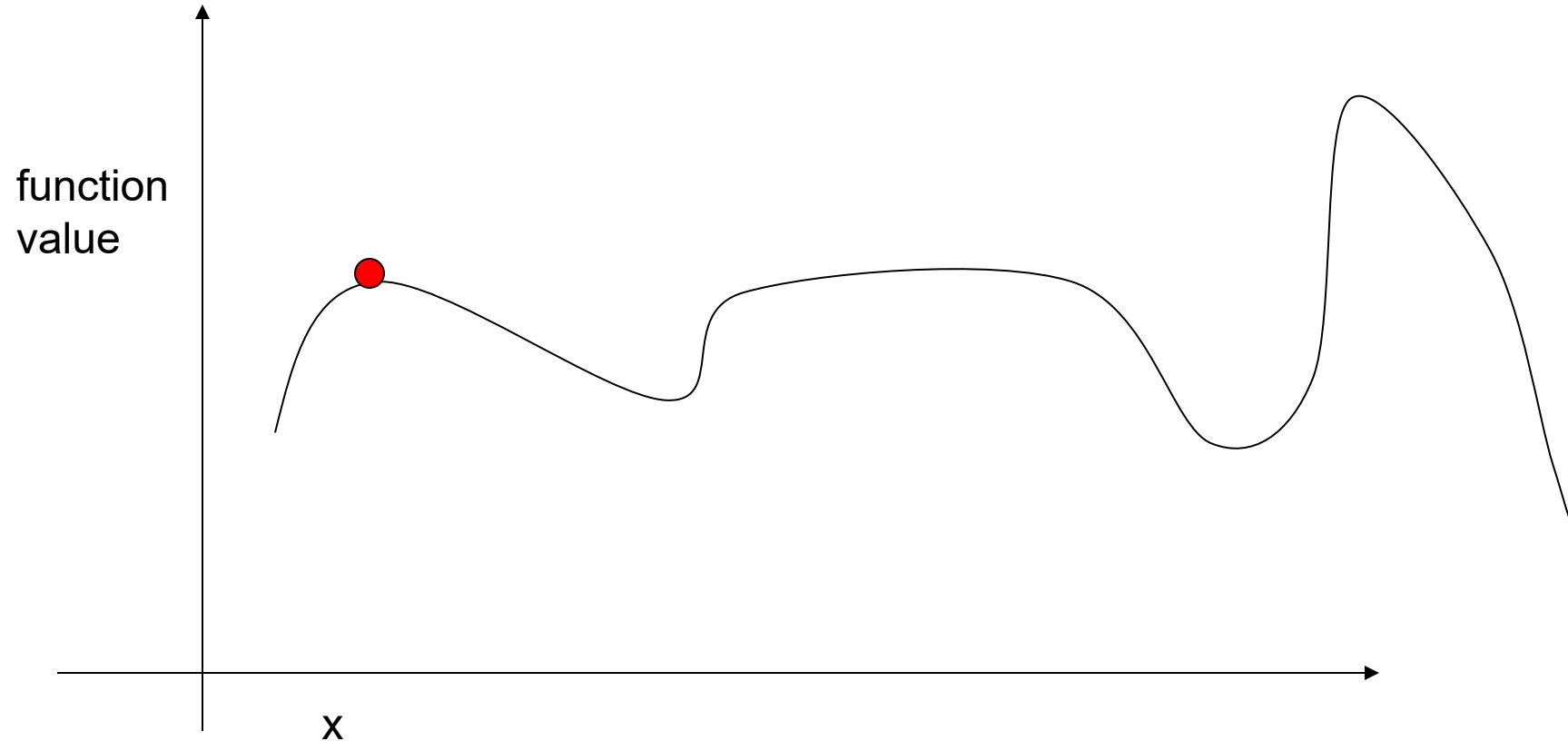
Gradient Ascent

- Could reduce step size to “hone in”



Gradient Ascent

- Converge to (local) maximum



Gradient Descent

- The basic equation that describes the update rule of gradient descent is:

Repeat Until Convergence {

$$\boxed{w} \leftarrow w - \boxed{\alpha} * \boxed{\nabla_w \sum_1^m L_m(w)}$$

}

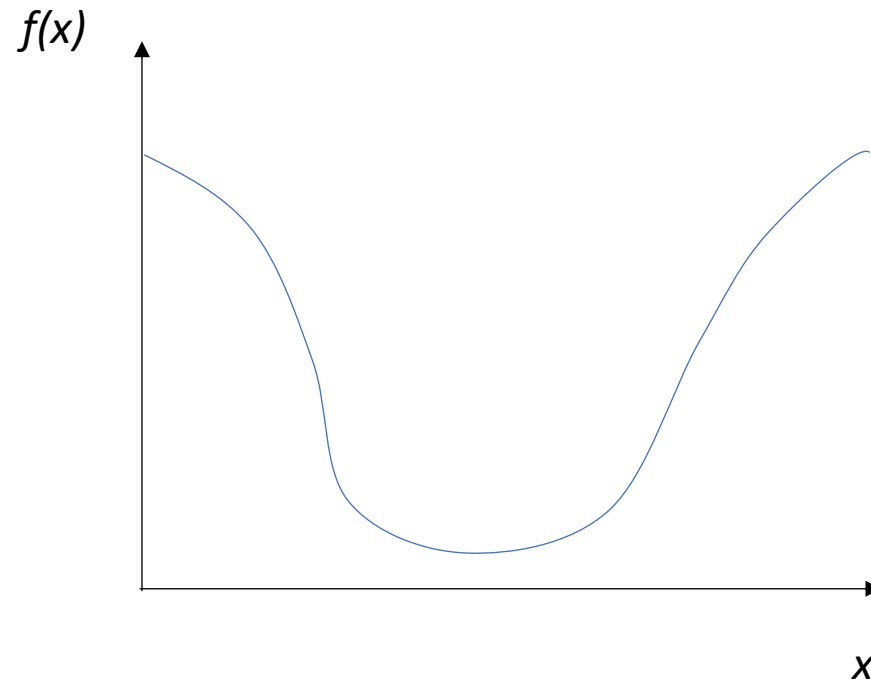
Learning Rate

- This update is performed during every iteration.
- Here, ``w'' is the weights vector, which lies in the x-y plane.
- From this vector, we subtract the gradient of the loss function with respect to the weights multiplied by alpha (or Gamma) which is referred to as the learning rate.
- The gradient is a vector which gives us the direction in which the loss function has the steepest ascent.
- The direction of the steepest descent is the direction exactly opposite to the gradient and that is why we are subtracting the gradient vector from the weights vector.

Gradient Descent

$$\theta^{t+1} = \theta^t - \eta \nabla L(\theta^t)$$

$$w_{x+1} = w_x - \gamma \nabla f(w_x)$$

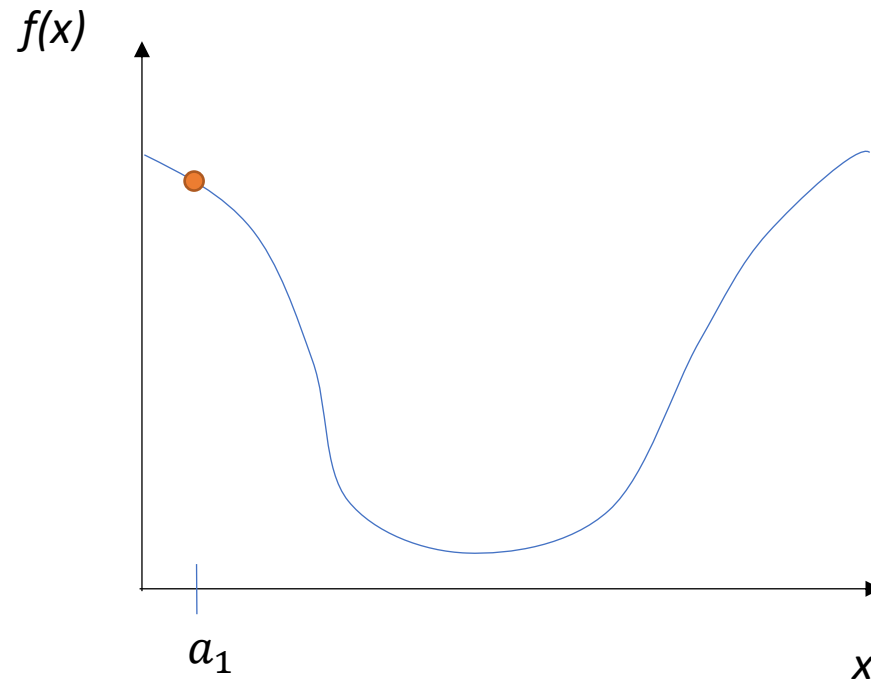


γ = amount to move
= *learning rate*

Gradient Descent

$$\theta^{t+1} = \theta^t - \eta \nabla L(\theta^t)$$

$$w_{x+1} = w_x - \gamma \nabla f(w_x)$$



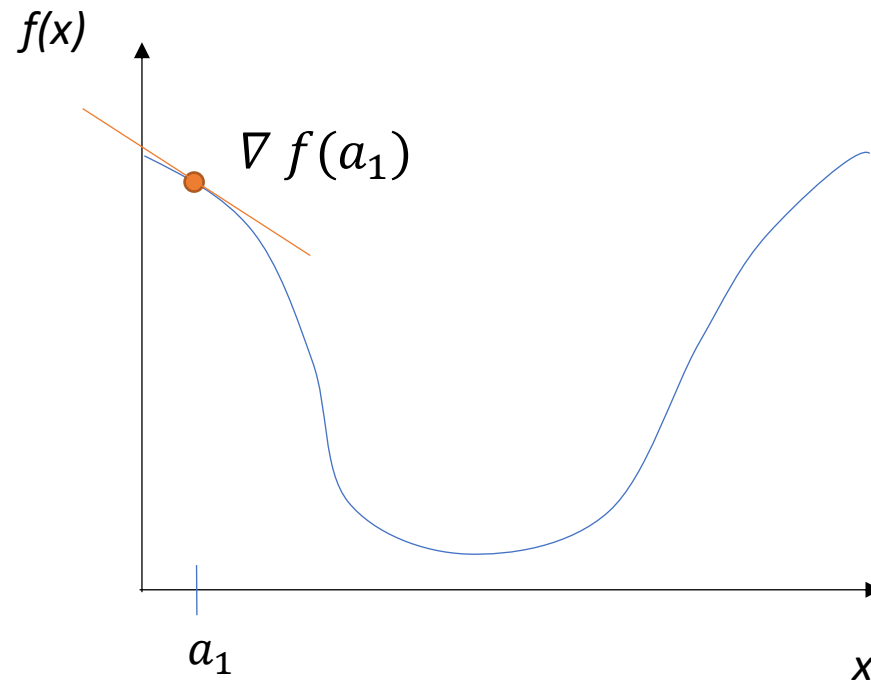
γ = amount to move
= *learning rate*

Pick random starting point.

Gradient Descent

$$\theta^{t+1} = \theta^t - \eta \nabla L(\theta^t)$$

$$w_{x+1} = w_x - \gamma \nabla f(w_x)$$



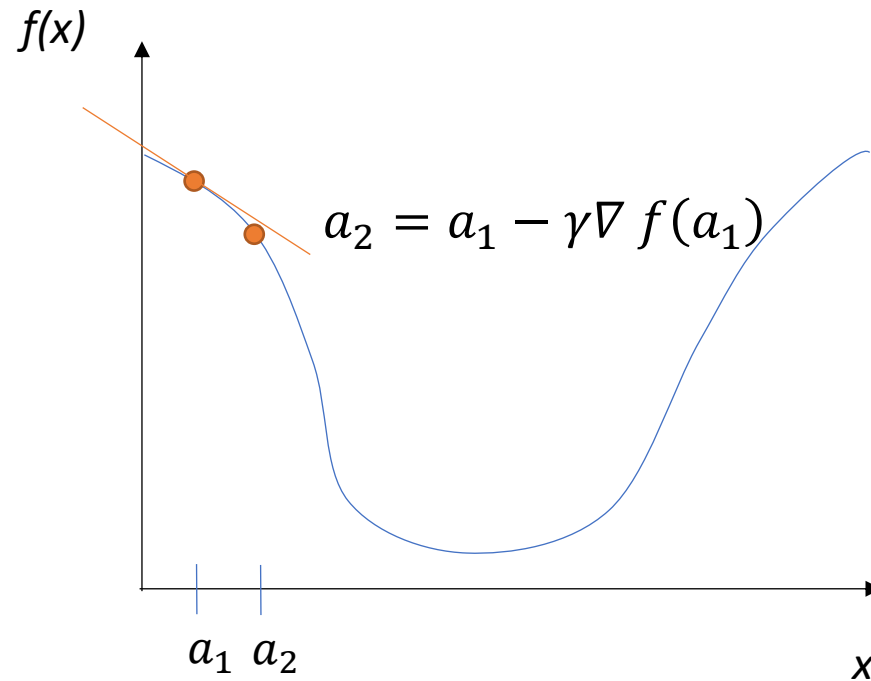
γ = amount to move
= *learning rate*

Compute gradient at point (analytically or by finite differences)

Gradient Descent

$$\theta^{t+1} = \theta^t - \eta \nabla L(\theta^t)$$

$$w_{x+1} = w_x - \gamma \nabla f(w_x)$$



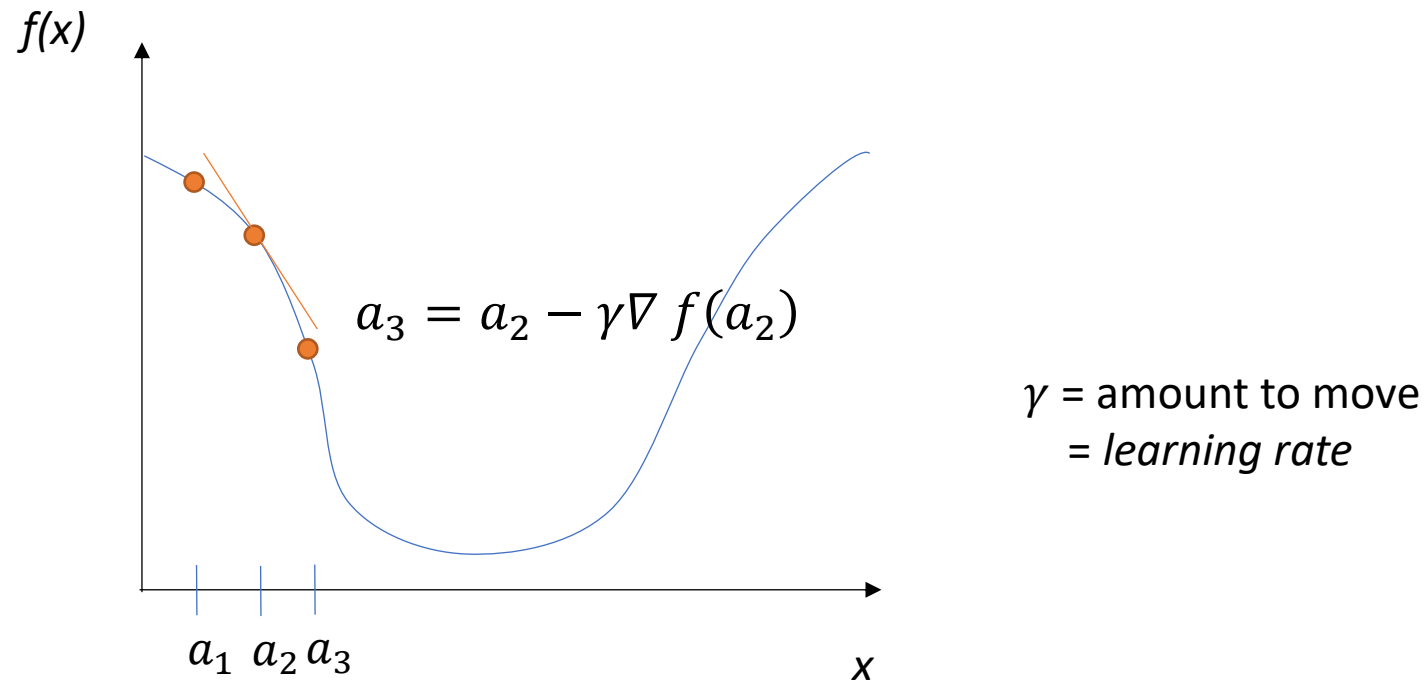
γ = amount to move
= *learning rate*

Move along parameter space in direction of negative gradient

Gradient Descent

$$\theta^{t+1} = \theta^t - \eta \nabla L(\theta^t)$$

$$w_{x+1} = w_x - \gamma \nabla f(w_x)$$



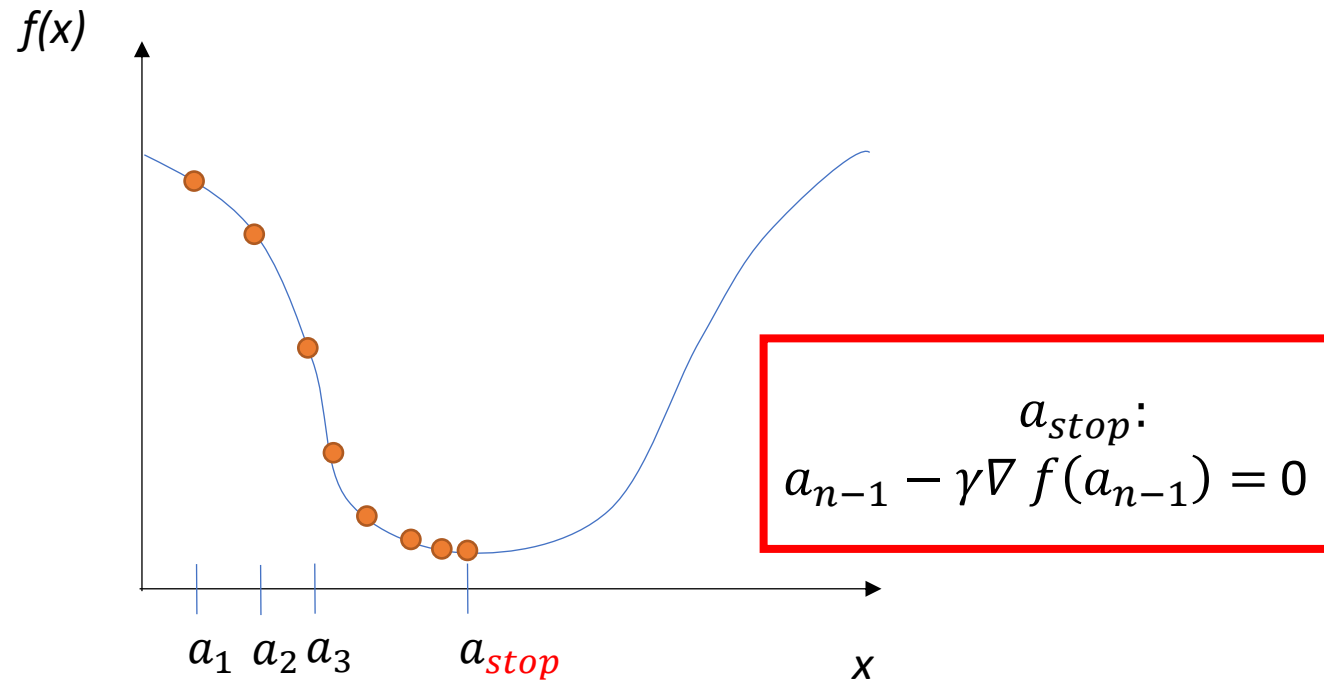
Move along parameter space in direction of negative gradient

Gradient Descent

Stop when we don't move any more.

The algorithm stops:

- When no further improvement can be achieved
- Or when a very small change in the value in parameter



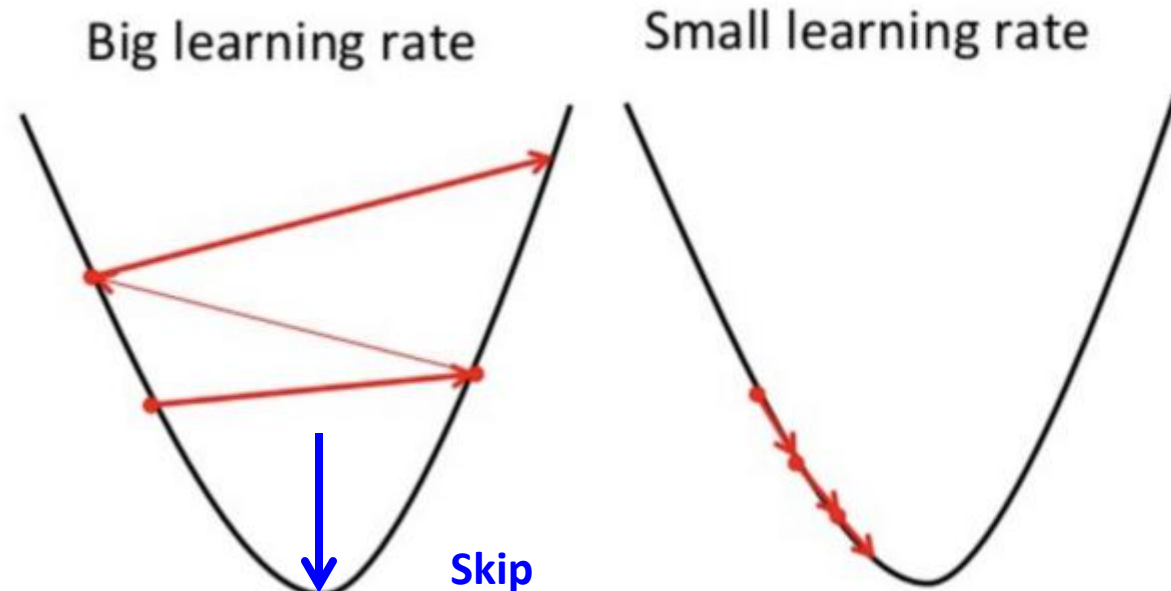
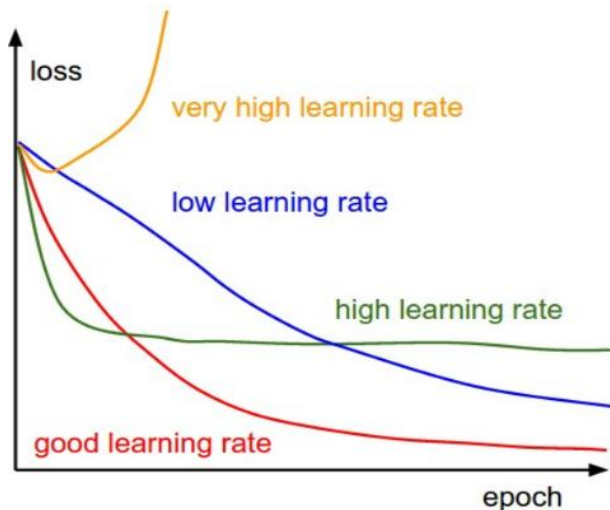
Gradient Descent Algorithm:

- 1. ****Initialize****: Choose an initial guess for the parameters (θ), denoted as ($\theta^{\{(0)\}}$), and set the iteration counter ($t = 0$).
- 2. ****Loop Until Convergence****:
 - - Compute the gradient of the objective function ($J(\theta)$) with respect to the parameters:
[$\nabla J(\theta^{\{(t)\}})$]
 - - Update the parameters using the gradient and the learning rate (α):
[$\theta^{\{(t+1)\}} = \theta^{\{(t)\}} - \alpha \nabla J(\theta^{\{(t)\}})$]
 - - Increment the iteration counter: ($t = t + 1$).
- 3. ****Convergence Criteria****:
 - - Check if the change in the parameters is below a certain threshold, or if the maximum number of iterations has been reached. If the convergence criteria are met, exit the loop.
- 4. ****Output****: The final parameters (θ) are the solution to the optimization problem.

Learning Rate

Learning rate must be chosen wisely (0.01 – 0.001) as:

1. If it is too small, then the model will take some time to converge (learn).
2. If it is too large, model will converge too quickly to a suboptimal solution ...as our pointer **will shoot** and we'll not be able to get to minima (will never learn!!)



What is a suitable value for Learning Rate? Static?

Gradient Descent: Issues

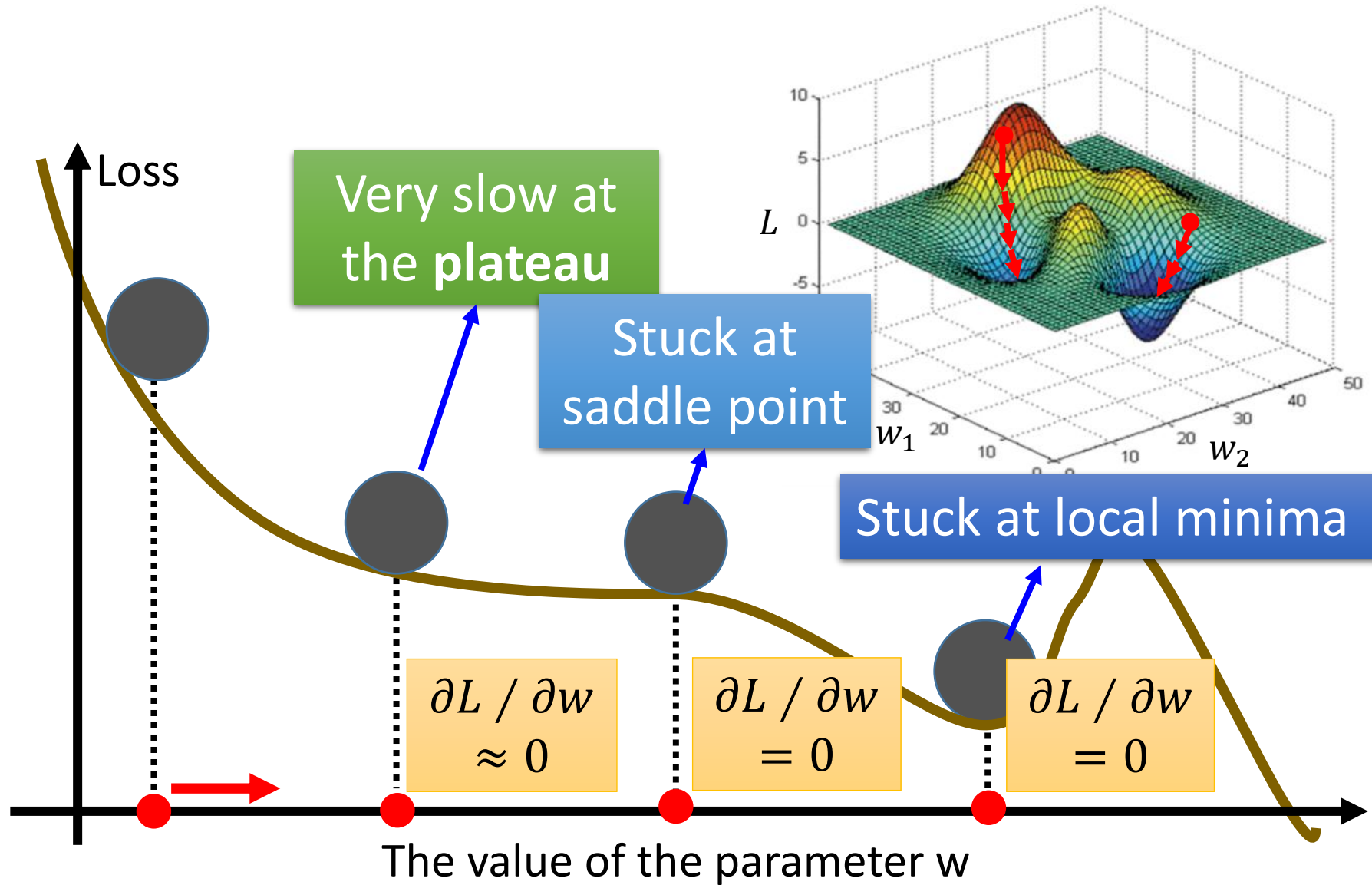
❖ Pros:

- Gradient Descent works well if we have a small number of data points.
- Computationally **efficient** since updates are required after the run of an epoch
- A more **direct path** is taken towards the minimum

❖ Cons:

- Can converge (get stuck!!) at **local minima** and saddle points
- Slower learning since an update is performed only after we go through all observations

Gradient Descent: Limitations



Saddle , Plateaus and Local points

- 1.saddle points are critical points with zero gradient but are not local minima.
- 2.Plateaus are regions of the search space with low gradient magnitude, which can slow down optimization algorithms.
- 3.Local minima are critical points where the function takes on the lowest value within a neighborhood, representing suboptimal solutions.

References

- [Linear Regression in Machine Learning](#)
- [Linear Regression in Machine learning – GeeksforGeeks](#)
- Linear Regression, ECE-5424G / CS-5824 Lecture Slides , Jia-Bin Huang , Virginia Tech
<https://filebox.ece.vt.edu/~jbhuang/teaching/ECE5424-CS5824/sp19/index.html>
- ENG6600: Advanced Machine Learning ,“Optimization for ML: Gradient Descent: (Part 1)”, S. Areibi ,School of Engineering University of Guelph
https://islab.soe.uoguelph.ca/sareibi/TEACHING_dr/ENG6600_ML_html_dr/outline_eng6600/docs/LECTURE_dr/TOPIC1_dr/LECT_dr/ENG6500-1-IntroductionToMLDL-Part1.pd
- [Stanford CS229: Machine Learning - Linear Regression and Gradient Descent | Lecture 2 \(Autumn 2018\)](#)
- [Linear Regression with One Variable | ML-005 Lecture 2 | Stanford University | Andrew Ng](#)
- Andrew, N. G. "Machine learning." *Coursera*. Available online: [*https://www.coursera.org/learn/machine-learning*](https://www.coursera.org/learn/machine-learning) (accessed on 1 March 2019) (2016).